

TuntiTutka

Paperinen työpaketti

Työaikaseuranta mainostoimistolle: Svelte, Express, SQLite ja julkaisu tuotantoon

18 työviikkoa · päivätön aikataulu · luovutus 18. työviikon perjantai

Tämä paketti on aikataulu ja tarkistuslista tilanteisiin, joissa sivusto ei ole auki. Projektipäiväkirja kirjoitetaan sivustolla ja viedään repositoryn project-docs-kansioon. Rasti tässä vihossa ei ole palautus: työ on aina Git-repositoryssa.

Aikataulu on päivätön: työviikko 1 on se viikko, jolla opiskelija aloittaa, ja projekti kestää 18 työviikkoa.

Julkiseen repositoryyn ei laiteta henkilötietoja, koulun tunnisteita eikä muiden nimiä. Tekijänimestä sovitaan ohjaajan kanssa.

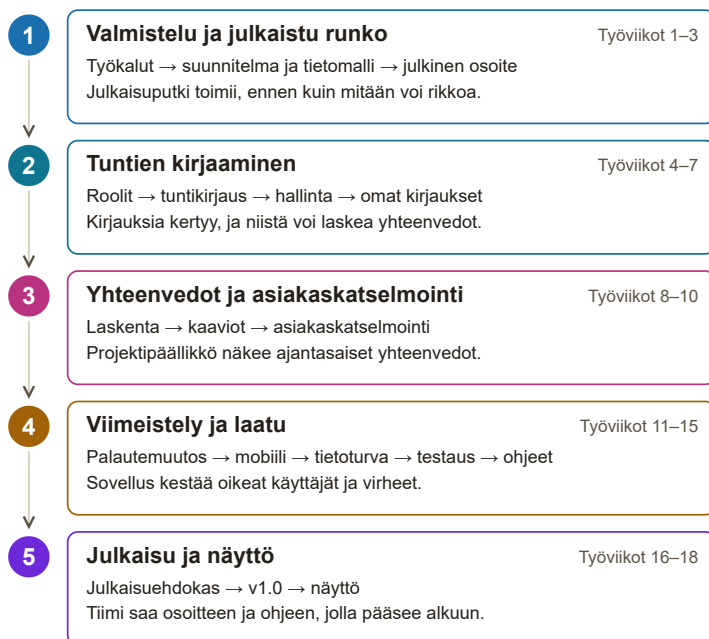
Mitä rakennat ja miten etenet

TuntiTutka on mainostoimiston sovellus työajan seurantaan. Siinä työntekijä kirjaa tuntinsa nopeasti myös puhelimella ja näkee vain omat kirjauksensa. Projektipäällikkö perustaa projektit ja näkee aina ajantasaiset yhteenvedot projekteittain, henkilöittäin ja tehtävälajeittain.

1. Työntekijä kirjaa tunnit puhelimella alle puolessa minuutissa ja näkee oman viikkosummansa.
2. Projektipäällikkö näkee pylväskaaviosta, montako tuntia kuhunkin projektiin kului valitulla viikolla.
3. Tehtävälajikaavio kertoo, mikä osuus ajasta meni suunnitteluun, tuotantoon, palaveriin ja korjauksiin.
4. Projektipäällikkö näkee tunnit henkilöittäin ja avaa yksittäisen työntekijän kirjaukset.

Projektin 5 vaihetta

Mitä syntyy missäkin vaiheessa?



1. Valmistelu ja julkaistu runko · työviikot 1–3. Otat työkalut käyttöön, teet hyväksytyn suunnitelman ja tietomallin ja viet tyhjän rungon julkiseen osoitteeseen. Tietomalli tehdään ensin, koska kaikki yhteenvedot lasketaan sen tauluista, ja julkaisu on helpointa korjata, kun rikottavaa on vähän.

2. Tuntien kirjaaminen · työviikot 4–7. Rakennat kirjautumisen ja roolit, tuntikirjauksen, projektien hallinnan ja omien kirjausten käsittelyn. Vaiheen lopussa työntekijät kirjaavat tunnit oikeisiin projekteihin ja sinä kirjaat omasi, joten raportteihin kertyy aitoa dataa.

3. Yhteenvedot ja asiakaskatselmointi · työviikot 8–10. Lasket kirjauksista yhteenvedot kolmella ryhmittelyllä ja teet niistä kaaviot. Asiakkaan roolissa oleva testaaja kokeilee väliversiota, ja hänen palautteensa ohjaa loppua.

4. Viimeistely ja laatu · työviikot 11–15. Toteutat katselmoinnin tärkeimmän muutoksen, teet sovelluksesta puhelimella ja näppäimistöllä toimivan ja arvioit tietoturvan. Testaus ja dokumentaatio kootaan valmiiksi ennen julkaisua.

5. Julkaisu ja näyttö · työviikot 16–18. Ulkopuolinen testaaja kokeilee julkaisuehdokasta pelkän kirjallisen ohjeen avulla. Korjaat estävät virheet, julkaiset v1.0:n ja kokoat näyttöaineiston.

Viikon työvaihe on tämän sivuston työohje. Kun toteutat muutoksen sovellukseen, kirjaa se GitHub-issuiksi hyväksymiskriteereineen ja tee se Työtapa-sivun kuudella askeleella. Testin tulos kirjataan issueen heti, viikon yhteenvedo projektipäiväkirjaan viikon lopussa.

Aikataulu

Vko	Ajoitus	Viikon aihe	Vaihe
1	Työviikko 1 / 18	Aloitus	1
2	Työviikko 2 / 18	Suunnitelma ja tietomalli	1
3	Työviikko 3 / 18	Julkaistu runko	1
4	Työviikko 4 / 18	Kirjautuminen ja roolit	2
5	Työviikko 5 / 18	Tuntikirjaus	2
6	Työviikko 6 / 18	Hallintanäkymät	2
7	Työviikko 7 / 18	Omat kirjaukset haltuun	2
8	Työviikko 8 / 18	Yhteenvedojen laskenta	3
9	Työviikko 9 / 18	Raporttinäkymä ja kaaviot	3
10	Työviikko 10 / 18	Asiakaskatselmointi	3
11	Työviikko 11 / 18	Palautemuutos	4
12	Työviikko 12 / 18	Mobiili ja saavutettavuus	4
13	Työviikko 13 / 18	Tietoturva	4
14	Työviikko 14 / 18	Testausviikko	4
15	Työviikko 15 / 18	Laatu ja dokumentointi	4
16	Työviikko 16 / 18	Julkaisuehdokas ja julkaisutestaus	5
17	Työviikko 17 / 18	Julkaisu v1.0	5
18	Työviikko 18 / 18	Näyttö	5

Palautus viimeistään 18. työviikon perjantai. Sivusto: tehtävien tarkat ohjeet, toteutusavut ja projektipäiväkirja.

Vaihe 1 – Valmistelu ja julkaistu runko

Työviikko 1 / 18 – Aloitus

Yhteys kokonaisprojektiin: Lähtöaineistona on toimeksianto, ja tällä viikolla syntyvät kehitysympäristö, julkinen repository ja käynnistynyt Svelte-runko. Git-historia on itsessään työnäyte, joten se aloitetaan ensimmäisenä työpäivänä. Kysymyslistan vastauksilla rajaat työviikolla 2 sen, mitä rakennetaan.

Viikon tavoite: Svelte-sovellusrunko käynnistyy omalla koneellasi README:n ohjeilla, ja julkinen repository on tarkistettu.

[GitHub Desktop: kuvaohje ilman Git-komentoja](#)

Työvaihe 1 / 4 • Asenna työkalut ja kirjaa versiot

- ☐ Asenna työkalut. Asenna Node LTS eli pitkään tuettu vakaa versio, VS Code sekä GitHub Desktop (viikon kuvaohje) tai komentorivin Git.
- ☐ Tarkista versiot. Aja `node -v` ja `npm -v`. Kirjaa lisäksi GitHub Desktopin versio kohdasta Help → About GitHub Desktop tai komentorivin Gitin versio komennolla `git --version`.
- ☐ Kirjaa versiotaulukko. Luo projektikansio `tuntitutka/` ja tee sen juureen `README.md`. Kirjoita tiedostoon taulukko: työkalu, versio ja tarkistuskomento, esimerkiksi Node 20.11.1 → `node -v`.

Valmis kun: `README.md`:ssä on taulukko, jossa jokaisella työkalulla on versio ja tarkistuskomento.

Tallenna työnäyte: `README.md` repositoryyn ensimmäisessä commitissa (työvaihe 4). Versiot myös työviikon 1 päiväkirjaan.

Työvaihe 2 / 4 • Perusta repository ja tee julkisuustarkistus

- ☐ Luo repository. Lisää nykyinen `tuntitutka`-kansio GitHub Desktopiin kuvaohjeen reitillä B. Jos oma repo on jo GitHubissa, kloonaa se reitillä C ja siirrä `README` siihen. Uuden paikallisen repon julkaiset työvaiheessa 4 julkisuustarkistusten jälkeen.
- ☐ Käy tarkistuslista läpi. Käy julkisen repon tarkistuslista läpi: ei henkilötietoja, ei koulun tunnisteita eikä muiden nimiä.
- ☐ Sovi tekijänimi. Sovi ohjaajan kanssa, millä nimellä esiinnyt repositoryssa. Jos olet alaikäinen, huoltajan suostumus hoidetaan ohjaajan kautta.
- ☐ Pyydä kiittäys. Pyydä ohjaajalta tarkistuslistaan kirjallinen kiittäys. Kirjaa kiittaamatta jäävät asiat avoimiksi asioiksi, kunnes ne on sovittu.

Valmis kun: Repository on olemassa, ja ohjaaja on kuitannut julkisen repon tarkistuslistan kirjallisesti.

Tallenna työnäyte: Projektikansion polku ja julkisuusasioiden tila työviikon 1 päiväkirjaan; lisää repositoryn osoite julkaisemisen jälkeen. Avoimiksi jääneet asiat suunnitelman OHJAAJA-kenttiin.

Työvaihe 3 / 4 • Luo Svelte-runko ja kansiorakenne

- ☐ Luo sovellusrunko. Luo Svelte + Vite -projekti kansioon `client/` komennolla `npm create vite@latest` (pohja svelte) ja aja `npm install`.
- ☐ Käynnistä kehityspalvelin. Aja `npm run dev` ja avaa sovellus selaimessa. Ota kuvakaappaus käynnistyneestä rungosta.
- ☐ Luo kansiorakenne. Luo kansiot `server/` ja `project-docs/`. Lisää `.gitignore`, joka estää kansion `node_modules/`, tietokantatiedostot `*.db` ja tiedoston `.env`.
- ☐ Kirjoita README. Kirjoita README:hen, mitä olet tekemässä, kenelle ja millä komennoilla projekti käynnistyy.

Valmis kun: `npm run dev` avaa rungon selaimessa, ja kansiot `client/`, `server/` ja `project-docs/` sekä `README` ja `.gitignore` ovat olemassa.

Tallenna työnäyte: Kuvakaappaus käynnistyneestä rungosta työviikon 1 päiväkirjaan. Kansiot ja README menevät repositoryyn työvaiheen 4 commitissa.

Työvaihe 4 / 4 • Kirjoita kysymyslista ja vie työ GitHubiin

- ☐ Lue toimeksianto. Lue toimeksianto läpi ja alleviivaa pakolliset asiat. Asiakkaan ydinkipu on raportointi, ei kirjaaminen.
- ☐ Kirjoita kysymykset. Kirjoita jokaisesta epäselvästä kohdasta kysymys ohjaajalle tiedostoon `project-docs/kysymykset.md`. Tavoite on vähintään kuusi kysymystä.
- ☐ Kysy lisenssistä. Ota kysymyslistaan mukaan lisenssi. Toimeksiannon mukaan se kysytään ohjaajalta jo työviikolla 1.
- ☐ Tee ensimmäinen commit. Tarkista Desktopin Changes-lista ja tee Commit kuvaavalla Summary-viestillä, esimerkiksi "Perusta projekti, kansiorakenne ja Svelte-runko". Katso kuvaohjeen kohta 5. Commit tallentaa Git-historiaan omalla koneellasi.
- ☐ Vie GitHubiin. Valitse Desktopissa Publish repository uudelle paikalliselle repolle ja sovittu julkisuus. Jo julkaistulle repolle valitse Push origin. Tarkista View on GitHub -toiminnolla, että tiedostot ja sisältöcommit näkyvät GitHubissa. Katso kuvaohjeen kohdat 6–7.

Valmis kun: Kysymyslistassa on vähintään kuusi kysymystä, ja ensimmäinen commit näkyy GitHubissa.

Tallenna työnäyte: `project-docs/kysymykset.md` repositoryyn. Ensimmäisen commitin tunnus ja kysymykset työviikon 1 päiväkirjaan.

Viikon lopputarkistus: Repositoryssa on käynnistytävä Svelte-runko ja README, jossa on käynnistyskomennot; ohjaaja on kuitannut julkisen repon tarkistuslistan; kysymyslistassa on vähintään kuusi kysymystä.

Työnäyte Git-repositoryyn ennen rastia: commit-historian alku, kuvakaappaus dev-palvelimesta selaimessa, kuitattu julkisuustarkistuslista ja kysymyslista (p1, k1; s12 alkaa).

Työviikko 2 / 18 – Suunnitelma ja tietomalli

Yhteys kokonaisprojektiin: Työviikon 1 kysymyslista saa nyt vastaukset, ja toimeksianto muuttuu priorisoiduiksi käyttäjätarinoiksi, tietomalliksi ja GitHub-issueiksi. Tietomallin päätös "lasketaan, ei tallenneta" ratkaisee jo nyt, ettei sovellukseen synny summatauluja, joita pitäisi pitää ajan tasalla. Hyväksytyn suunnitelman varaan rakennat työviikolla 3 tietokannan ja julkaistun rungon.

Viikon tavoite: Ohjaaja hyväksyy suunnitelmasi: jokaisella pakollisen ytimen (P0) tarinalla on GitHub-issue, hyväksymiskriteerit ja työmääräarvio.

[Ava suunnitelmalomake](#)

Työvaihe 1 / 4 • Kirjoita käyttäjätarinat ja priorisoi ne

- ☐ Kirjaa ohjaajan vastaukset. Käy työviikon 1 kysymyslista läpi ohjaajan kanssa ja kirjaa jokainen vastaus kysymyksen alle omin sanoin.
- ☐ Kirjoita tarinat. Kirjoita toimeksiannosta käyttäjätarinat muodossa "Työntekijänä ... jotta ..." tai "Projektipäällikkönä ... jotta ...". Anna jokaiselle 2–4 hyväksymiskriteeriä.
- ☐ Priorisoi ohjaajan kanssa. Merkitse tarinat ohjaajan kanssa, joka toimii asiakkaan sijaisena. P0 on pakollinen ydin, P1 on tärkeä jatko ja P2 on valinnainen lisä.
- ☐ Kirjaa perustelut. Kirjoita jokaisen prioriteetin viereen perustelu, älä pelkkää kirjainta. Täytä suunnitelmaan tavoite ja käyttäjäryhmät omin sanoin.

Valmis kun: Jokaisella tarinalla on 2–4 hyväksymiskriteeriä ja perusteltu prioriteetti, ja ohjaajan vastaukset on kirjattu kysymyslistaan.

Tallenna työnäyte: Käyttäjätarinat ja kysymyslista vastauksineen `project-docs/-kansioon`. P0-rajaus työviikon 2 päiväkirjaan.

Työvaihe 2 / 4 · Piirrä tietomalli ja kirjaa laskentaperiaate

- ☐ Piirrä taulut. Piirrä taulut käyttäjä, projekti, projektityyppi, tehtävätyyppi, kirjaus ja projektin_jasen. Merkitse pääavaimet, viiteavaimet ja pakolliset kentät.
- ☐ Kirjaa periaate. Kirjoita kaavion viereen periaate: yhteenvetosummaa ei tallenneta, vaan ne lasketaan kirjauksista jokaisella pyynnöllä.
- ☐ Tarkista tarinoita vasten. Käy jokainen pakollisen ytimen (P0) tarina läpi mallia vasten: löytyykö jokaiselle hyväksymiskriteerille kenttä tai taulu? Jos ei, täydennä mallia.
- ☐ Kirjaa suunnitelmaan. Kirjoita taulut, avaimet ja se, mitä ei tallenneta, suunnitelman Tietomalli-kenttään.

Valmis kun: Tietomallikaaviossa on kaikki kuusi taulua avaimineen, ja periaate ”summaa ei tallenneta” on kirjattu kaavioon ja suunnitelmaan.

Tallenna työnäyte: Tietomallikaavio kuvana project-docs/-kansioon. Taulut ja tallentamatta jätettävät summat työviikon 2 päiväkirjaan.

Työvaihe 3 / 4 · Vertaa tietovarastoja ja perustele valinta

- ☐ Tee vertailutaulukko. Vertaa SQLiteä, JSON-tiedostoa ja PostgreSQLä. JSON on tekstimuotoinen tietomuoto, jossa tieto on nimi-arvo-pareina.
- ☐ Käytä omia kriteereitä. Arvioi vaihtoehdot oman datasi rakenteen, käyttötilanteen ja laajuuden perusteella, älä yleisten ominaisuuksien.
- ☐ Perustele valinta. Kirjoita valinta ja se, mikä valinnasta jää huonommaksi, suunnitelman Tietovarasto-kenttään.
- ☐ Tarkista sovittu toteutustapa. Toimeksianto sopii SQLiten. Jos vertailusi puoltaa muuta, sovi asiasta ohjaajan kanssa ennen työviikkoa 3.

Valmis kun: Vertailutaulukossa on kolme vaihtoehtoa omilla kriteereilläsi, ja valinta on perusteltu suunnitelmassa omilla tauluillasi.

Tallenna työnäyte: Vertailutaulukko project-docs/-kansioon. Valinta perusteluineen työviikon 2 päiväkirjaan.

Työvaihe 4 / 4 · Piirrä rautalangat ja pilko P0 issueiksi

- ☐ Piirrä rautalangat. Piirrä kirjausnäkö ja raporttinäkö käsin tai piirtotyökalulla. Rautalanka on yksinkertainen näkymäluonnos ilman värejä. Merkitse kummankin kentät ja toiminnot.
- ☐ Pilko issueiksi. Pilko pakollisen ytimen (P0) tarinat GitHub-issueiksi. Yksi issue on tyypillisesti puolen tai yhden päivän työ.
- ☐ Lisää kriteerit ja arviot. Kirjoita jokaiseen issueen hyväksymiskriteerit ja työmääräarvio tunteina. Arvioihin palataan työviikolla 7.
- ☐ Vie suunnitelma repositoryyn. Täytä Suunnitelma-näkymän kentät, lataa suunnitelma.md ja tallenna se polkuun project-docs/suunnitelma.md commitilla.
- ☐ Pyydä hyväksyntä. Pyydä ohjaajalta kirjattu hyväksyntä rajaukselle, esimerkiksi kommenttina issueen. Suunnitelma hyväksytään ennen työviikkoa 3.

Valmis kun: Jokaisella P0-tarinalla on issue, hyväksymiskriteerit ja arvio tunteina, ja ohjaajan hyväksyntä on kirjattu.

Tallenna työnäyte: Rautalangat ja suunnitelma.md project-docs/-kansioon. Linkki issue-tilaan ja ohjaajan hyväksyntään työviikon 2 päiväkirjaan.

Viikon lopputarkistus: project-docs/suunnitelma.md on repossa ja ohjaaja on hyväksynyt rajauksen kirjatulla kommentilla; jokaisella P0-tarinalla on issue, hyväksymiskriteerit ja arvio tunteina.

Työnäyte Git-repositoryyn ennen rastia: suunnitelma.md, tietomallikaavio, vertailutaulukko, issue-tila ja ohjaajan kirjattu hyväksyntä (s1, s4, s5, s8, p8; s6 alkaa arvioista).

Työviikko 3 / 18 – Julkaistu runko

Yhteys kokonaisprojektiin: Työviikon 2 tietomalli muuttuu nyt ajettavaksi skeemaksi, ja sovellus saa palvelimen. Koko julkaisuputki testataan tällä viikolla, koska se on helpointa korjata, kun sovelluksessa ei vielä ole mitään rikkoutuvaa. Samaa julkaisuputkea käytät julkaisuehdokkaaseen ja v1.0:aan asti.

Viikon tavoite: Kuka tahansa voi avata TuntiTutkan julkisesta osoitteesta, ja osoite /api/health vastaa toisen henkilön laitteella.

Työvaihe 1 / 4 • Rakenna Express-palvelin ja health-reitti

- ☐ Tee palvelin. Rakenna `server/index.js`, joka tarjoaa Sveltin tuotantobuildin staattisena. Build on valmiiksi käännetty, julkaistava versio sovelluksesta.
- ☐ Lisää health-reitti. Tee reitti `/api/health`, joka palauttaa JSONin, esimerkiksi `{ tila: "ok" }`, ja HTTP-koodin 200. Se on oman rajapintasi (API) ensimmäinen osoite.
- ☐ Kytke dev-proxy. Lisää Viten asetuksiin proxy eli välitysasetus, joka ohjaa kehityspalvelimen `/api`-kutsut Express-palvelimelle.
- ☐ Kokeile molemmat. Avaa `/api/health` sekä Expressin osoitteessa että Viten kehityspalvelimen kautta. Saman vastauksen pitää näkyä molemmissa.

Valmis kun: `/api/health` palauttaa 200 ja JSONin sekä Expressin osoitteessa että Viten proxy:n kautta.

Tallenna työnäyte: `server/index.js`, reitti ja proxy-asetus commitilla repositoryyn.

Työvaihe 2 / 4 • Luo SQLite-skeema init-skriptillä

- ☐ Kirjoita init-skripti. Skeema tarkoittaa tietokannan taulurakennetta. Kirjoita `server/init-db.js`, joka luo työviikon 2 tietomallin taulut tyhjään tietokantaan.
- ☐ Keskitä yhteys. Tee tietokantayhteys yhteen paikkaan, esimerkiksi `server/db.js`, jotta reitit eivät avaa omia yhteyksiään.
- ☐ Aja ja tarkista. Aja skripti tyhjään tietokantatiedostoon ja tarkista, että kaikki taulut syntyivät eikä summataulua ole.

Valmis kun: Init-skripti luo tyhjään tietokantaan kaikki tietomallin taulut, eikä tietokantatiedosto ole repositoryssa.

Tallenna työnäyte: `server/init-db.js` ja `server/db.js` commitilla. Skeema ja init-skripti pohjustavat tietovarastoyhteyden työnäytettä (s9).

Työvaihe 3 / 4 • Vertaa julkaisualustat ja perustele valinta

- ☐ Tarkista oppilaitoksen linja. Kysy ohjaajalta oppilaitoksen linja ilmaisista pilvipalveluista, jos se on vielä avoin. Linja rajaa valintaasi.
- ☐ Vertaa alustoja. Vertaa Renderiä, Flyiota ja Railwayta taulukkona: ilmainen taso, Node-tuki, SQLite-tiedoston pysyvyys ja levy sekä lokit.
- ☐ Perustele valinta. Valitse alusta ja kirjoita perustelu suunnitelman Julkaisualusta-kenttään. Tämä on oma päätöksesi.

Valmis kun: Alustavertailu on taulukkona, ja valinta on perusteltu suunnitelmassa, ennen kuin loit tilin mihinkään palveluun.

Tallenna työnäyte: Alustavertailu `project-docs/-kansioon`. Valittu alusta ja valintakriteerit työviikon 3 päiväkirjaan.

Työvaihe 4 / 4 • Julkaise runko ja kirjoita Svelte-selvitys

- ☐ Tarkista asetukset. Varmista, että palvelin kuuntelee alustan antamaa porttia (PORT), tietokantatiedoston polku on ympäristömuuttujassa eikä `.env` ole repositoryssa.
- ☐ Julkaise. Vie runko valitulle alustalle ja kirjaa julkinen osoite README:hen. Ota kuvakaappaus alustan julkaisulokista.
- ☐ Tee savutesti. Savutesti on nopea tarkistus, että perustoiminto vastaa julkaistussa versiossa. Pyydä toista henkilöä avaamaan `/api/health` omalla laitteellaan ja kirjaa tulos.
- ☐ Kirjoita selvitys. Kirjoita taulukkona, mitä Svelte ja Vite ratkaisevat, mitä eivät ja mistä puuttuva otetaan. Tämä on osaamisvaatimuksen k2 työnäyte.

Valmis kun: `/api/health` palauttaa 200 julkisessa osoitteessa toisen henkilön laitteella, ja selvitys ja alustaperustelu ovat repositoryssa.

Tallenna työnäyte: Julkinen osoite README:hen, julkaisulokin kuvakaappaus ja selvitys `project-docs/-kansioon`. Mitä julkaisussa meni pieleen ja miten korjasit sen, työviikon 3 päiväkirjaan.

Viikon lopputarkistus: `/api/health` palauttaa 200 julkisessa osoitteessa toisen henkilön selaimella testattuna; init-skripti luo skeeman tyhjään tietokantaan; k2-selvitys ja alustaperustelu ovat repossa.

Työnäyte Git-repositoryyn ennen rastia: julkinen osoite README:ssä, julkaisulokin kuvakaappaus, init-skripti, k2-selvitys ja alustavertailu (k2, s14 ensijulkaisu; s9 pohjustuu).

Vaihe 2 – Tuntien kirjaaminen

Työviikko 4 / 18 – Kirjautuminen ja roolit

Yhteys kokonaisprojektiin: Julkaistun rungon päälle tulee ensimmäinen oikea toiminto: käyttäjät ja roolit. Kirjaukset, hallinta ja raportit rajataan roolilla, joten roolit tehdään nyt, eikä jokaista reittiä ja näkymää tarvitse myöhemmin avata uudelleen. Työviikolla 5 kirjautunut työntekijä kirjaa ensimmäiset tuntinsa.

Viikon tavoite: Työntekijä ja projektipäällikkö kirjautuvat sisään ja näkevät roolinsa mukaisen navigaation, ja suojattu reitti vastaa ilman kirjautumista koodilla 401.

Työvaihe 1 / 4 • Toteuta käyttäjätaulu ja siemendata

- ☐ Tee käyttäjätaulu. Toteuta käyttäjä-taulu, jossa on nimi, sähköposti, salasanan hash, rooli ja puhelin.
- ☐ Hashaa salasanat. Tallenna salasanat bcrypt-hashina. Hash on salasanasta laskettu tiiviste, josta alkuperäistä salasanaa ei saa takaisin. Selväkielistä salasanaa ei kirjoiteta lokiin eikä kantaan.
- ☐ Luo siemendata. Luo skripti, joka lisää yhden projektipäällikön ja kaksi työntekijää. Käytä keksittyjä tunnuksia, ei oikeiden ihmisten tietoja.

Valmis kun: Tietokannassa on yksi projektipäällikkö ja kaksi työntekijää, eikä yhtään salasanaa ole tallennettu selväkielisenä.

Tallenna työnäyte: Taulu ja siemenskripti commitilla repositoryyn.

Työvaihe 2 / 4 • Vertaa istuntotapoja ja päätä ohjaajan kanssa

- ☐ Vertaa tapoja. Vertaa evästesessiota ja tokenia eli selaimelle annettavaa tunnistetietoa juuri tässä sovelluksessa: SQLite ja yksi palvelin. Kirjoita molempien hyödyt ja riskit.
- ☐ Päätä yhdessä. Käy vertailu läpi ohjaajan kanssa ennen toteutusta ja tee päätös yhdessä.
- ☐ Kirjaa keskustelu. Kirjaa muistioon päätös, perustelu, keskustelukumppani, hänen roolinsa ja ajankohta. Täytä suunnitelman Istuntotapa-kenttä.

Valmis kun: Vertailu on kirjoitettu tämän sovelluksen ehdoilla, ja yhteinen päätös on kirjattu ohjaajan nimen ja ajankohdan kanssa.

Tallenna työnäyte: Vertailu ja muistio project-docs/-kansioon. Päätös perusteluineen työviikon 4 päiväkirjaan.

Työvaihe 3 / 4 • Toteuta kirjautuminen ja roolin mukainen navigaatio

- ☐ Tee kirjautuminen. Toteuta kirjautumislomake, kirjautuminen ja uloskirjautuminen valitsemallasi istuntotavalla.
- ☐ Välitä rooli. Välitä roolitieto selaimen, esimerkiksi storeen `client/src/lib/istunto.js`. Kirjautuminen säilyy, kun sivu päivitetään.
- ☐ Näytä virheet. Väärä salasana antaa selkeän suomenkielisen virheilmoituksen kentän vieressä.
- ☐ Rajaa navigaatio. Näytä hallintalinkit vain projektipäällikölle. Piilotettu linkki ei ole suojas: reitit suojataan palvelimella työvaiheessa 4.

Valmis kun: Molemmat roolit kirjautuvat sisään ja ulos, kirjautuminen säilyy sivun päivityksessä, eikä työntekijä näe hallintalinkkejä.

Tallenna työnäyte: Kuvakaappaukset navigaatiosta molemmilla rooleilla työviikon 4 päiväkirjaan. Koodi commitilla.

Työvaihe 4 / 4 • Suojaa API-reitit middlewarella

- ☐ Tunnista käyttäjä. Middleware on välikerros, joka tarkistaa pyynnön ennen reitin omaa käsittelijää. Kirjoita autentikointimiddleware, joka palauttaa 401, jos voimassa olevaa istuntoa ei ole.
- ☐ Tarkista rooli. Kirjoita roolimiddleware, joka palauttaa 403, jos käyttäjällä ei ole reitin vaatimaa roolia.
- ☐ Suojaa jokainen reitti. Lisää middleware jokaiselle suojatulle reitille, älä vain navigaatioon.
- ☐ Testaa molemmilla rooleilla. Kirjaudu kummallakin roolilla ja tarkista, että kumpikin näkee vain omat näkymänsä.
- ☐ Kutsu reittiä suoraan. Kutsu suojattua reittiä curlilla ilman kirjautumista ja työntekijän istunnolla. curl on komentorivin työkalu, jolla rajapintaa kutsutaan ilman selainta. Kirjaa saadut koodit.

Valmis kun: Suora kutsu suojattuun reittiin ilman kirjautumista palauttaa 401, ja työntekijän kutsu projektipäällikön reittiin palauttaa 403.

Tallenna työnäyte: Middleware-koodin commit. Mitä middleware-ketju tarkistaa, missä järjestyksessä ja mitkä koodit sait, työviikon 4 päiväkirjaan.

Viikon lopputarkistus: Väärä salasana antaa selkeän suomenkielisen virheilmoituksen; työntekijä ei näe hallintanavigaatiota; kirjautuminen säilyy sivun päivityksessä; suora API-kutsu ilman kirjautumista palauttaa 401.

Työnäyte Git-repositoryyn ennen rastia: kirjattu vertailu ja yhteinen päätös, kuvakaappaukset molemmilla rooleilla ja middleware-koodin commit (p9; s7 osa).

Työviikko 5 / 18 – Tuntikirjaus

Yhteys kokonaisprojektiin: Työviikolla 4 syntyneet kirjautuminen ja roolit kertovat, kuka kirjaa. Nyt rakennat itse kirjauksen: lomakkeen, tallennusreitit ja omien kirjausten listan. Kaikki myöhemmät yhteenvedot lasketaan näistä riveistä, joten tämän viikon validointi ratkaisee, voiko raportteihin luottaa.

Viikon tavoite: Työntekijä kirjaa tunnit projektille ja tehtävalajille alle 30 sekunnissa ja näkee kirjauksen omassa listassaan.

Työvaihe 1 / 4 · Rakenna kirjauslomake omista Svelte-komponenteista

- ☐ Päätä komponenttijako. Nimeä kirjausnäytteen komponentit ja kirjoita kunkin vastuu yhdellä lauseella suunnitelman Komponenttijako-kenttään. Jakoon palataan työviikolla 15.
- ☐ Rakenna lomake rautalangan mukaan. Toteuta kirjauslomake työviikon 2 rautalangan mukaan kansioon `client/src/kirjaus/`. Kentät ovat projekti, tehtävalaji, tunnit, päivä ja selite. Valmista käyttöliittymäkirjasto ei käytetä.
- ☐ Kytke kentät sidonnoilla. Käytä Sveltin sidontoja (`bind:value`) lomakekenttiin, jotta lomakkeen tila on aina sama kuin kentissä näkyy.

Valmis kun: Lomake näkyy selaimessa kaikkine kenttineen, ja komponenttijako on kirjattu suunnitelmaan.

Tallenna työnäyte: Komponentit commitilla kansioon `client/src/kirjaus/`. Komponenttijakokaavio ja kuvakaappaus lomakkeesta työviikon 5 päiväkirjaan.

Työvaihe 2 / 4 · Toteuta tallennusreitti ja validointi

- ☐ Tee tallennusreitti. Toteuta `POST /api/kirjaukset` tiedostoon `server/routes/kirjaukset.js`. REST-tyylissä osoite kertoo kohteen ja HTTP-metodi toiminnon: `POST` lisää ja `GET` hakee.
- ☐ Validoi palvelimella. Tarkista palvelimella, että tunnit ovat välillä 0,25–24, pakolliset kentät on täytetty sekä projekti ja tehtävalaji ovat olemassa.
- ☐ Validoi myös lomakkeessa. Toteuta samat säännöt lomakkeeseen. Pelkkä selaimen validointi ei riitä, koska rajapintaa voi kutsua suoraan.
- ☐ Näytä virhe kentän vieressä. Näytä jokainen virhe suomeksi sen kentän vieressä, jota virhe koskee. Viesti kertoo, mitä pitää tehdä, esimerkiksi "Tunnit voivat olla enintään 24."

Valmis kun: Kelvollinen kirjaus tallentuu SQLiteen. Virheellinen syöte hylätään sekä lomakkeessa että palvelimella, ja suomenkielinen viesti näkyy kentän vieressä.

Tallenna työnäyte: Reitti ja validointi commitilla. Validointisäännöt taulukkona (kenttä → sääntö → virheviesti) työviikon 5 päiväkirjaan.

Työvaihe 3 / 4 · Näytä omat kirjaukset listana

- ☐ Tee hakureitti. Toteuta `GET /api/kirjaukset/omat`, joka palauttaa vain kirjautuneen käyttäjän kirjaukset.
- ☐ Rakenna lista. Näytä omat kirjaukset taulukkona omassa komponentissaan, esimerkiksi `KirjausLista.svelte`.
- ☐ Tarkista pysyvyys. Päivitä sivu ja tarkista, että kirjaukset näkyvät yhä listassa. Ne tulevat tietokannasta, eivät selaimen muistista.

Valmis kun: Omat kirjaukset näkyvät listassa myös sivun päivityksen jälkeen, eikä listassa näy muiden käyttäjien kirjauksia.

Tallenna työnäyte: Reitti ja lista commitilla. Kuvakaappaus lomakkeesta ja listasta työviikon 5 päiväkirjaan.

Työvaihe 4 / 4 • Testaa virhesyötteet ja mittaa kirjausaika

- ☐ Kirjaa odotukset. Tee taulukko, jossa on syöte, odotettu tulos, havaittu tulos ja täsmääkö. Kirjoita odotettu tulos ennen kuin kokeilet.
- ☐ Testaa virhesyötteet. Kokeile lomakkeella arvoja 0 h, negatiivinen, 25 h, ei-numeerinen ja puuttuva tehtävälaji. Kirjaa jokaisen havaittu tulos.
- ☐ Ohita lomake. Lähetä `tunnit=25` suoraan rajapintaan curlilla. curl on komentorivin työkalu, jolla rajapintaa kutsutaan ilman selainta. Palvelimen pitää hylätä kirjaus.
- ☐ Mittaa kirjausaika. Ota aika yhdestä kirjauksesta alusta loppuun ja kirjaa sekunnit. Tavoite on alle 30 sekuntia.

Valmis kun: Jokainen virhesyöte hylätään sekä lomakkeessa että suorassa rajapintakutsussa, ja kirjaus onnistuu alle 30 sekunnissa.

Tallenna työnäyte: Testitaulukko ja mitattu kirjausaika sekunteina työviikon 5 päiväkirjaan.

Viikon lopputarkistus: Kirjaus tallentuu SQLiteen ja näkyy listassa sivun päivityksen jälkeenkin; virheellinen syöte ei tallennu ja käyttäjä näkee suomenkielisen virheviestin kentän vieressä; kirjaus onnistuu alle 30 sekunnissa. Ota aika.

Työnäyte Git-repositoryyn ennen rastia: komponenttijakokaavio, kuvakaappaus lomakkeesta ja listasta, validointitestien tulostaulukko ja commitit (p6, s9, k3; k5 toteutus alkaa).

Työviikko 6 / 18 – Hallintanäkymät

Yhteys kokonaisprojektiin: Kirjauslomake tarvitsee projekteja ja tehtävälajeja, ja ilman hallintaa ne pitäisi syöttää tietokantaan käsin. Nyt projektipäällikkö perustaa ne itse ja päättää jäsenyyksillä, kuka kirjaa mihinkin projektiin. Jäsenyys rajaa kirjaukset oikeisiin projekteihin, mikä on raporttien oikeellisuuden ehto.

Viikon tavoite: Projektipäällikkö perustaa projektin ja liittää siihen työntekijän, ja työntekijä näkee kirjauslomakkeessa vain omat projektinsa.

Työvaihe 1 / 4 • Valitse reitituskirjasto ja jaa sovellus reitteihin

- ☐ Vertaa vaihtoehtoja. Vertaa kahta reitituskirjastoa, esimerkiksi svelte-routing ja page.js: koko, ylläpito, dokumentaatio ja se, mitä juuri tämä projekti tarvitsee.
- ☐ Perustele ja asenna. Kirjaa valinta perusteluineen suunnitelman Reitituskirjasto-kenttään ennen asennusta. Asenna sitten valittu kirjasto.
- ☐ Kirjaa package.json-diffi. Tallenna package.json-diffi eli se, mitä riippuvuuksia asennus lisäsi.
- ☐ Tee reittikartta. Kirjoita taulukko URL → näkymä → sallittu rooli jokaiselle reitille ja jaa sovellus näihin näkymiin.

Valmis kun: Valinta on perusteltu suunnitelmassa, kirjasto on asennettu, ja jokaisella reittikartan rivillä on näkymä ja sallittu rooli.

Tallenna työnäyte: Kirjastovertilu ja reittikartta `project-docs/-`-kansioon, package.json-diffi commitissa.

Työvaihe 2 / 4 • Toteuta projektien hallinta (CRUD)

- ☐ Tee projektien reitit. CRUD tarkoittaa tietueen neljää perustoimintoa: luonti, luku, muokkaus ja poisto. Toteuta projekteille nämä reitit ja suoja ne projektipäällikön roolilla.
- ☐ Rakenna hallintanäkymä. Rakenna projektien listaus sekä luonti, muokkaus ja poisto. Projektilla on nimi, tyyppi, alku- ja loppupäivä sekä kuvaus.
- ☐ Kokeile hallintaa. Perusta testiprojekti, muokkaa sen aikaväliä ja poista se. Tarkista jokaisen vaiheen jälkeen, että lista päivittyy.

Valmis kun: Projektipäällikkö voi luoda, listata, muokata ja poistaa projekteja hallintanäkymässä ilman tietokannan käsin muokkausta.

Tallenna työnäyte: Reitit ja näkymä commitilla. Kuvakaappaus hallintanäkymästä työviikon 6 päiväkirjaan.

Työvaihe 3 / 4 · Toteuta tehtävälajien ja projektityyppien hallinta

- ☐ Toteuta lisäys. Toteuta tehtävälajin ja projektityypin lisäys hallintanäkymään.
- ☐ Toteuta muokkaus. Toteuta nimen ja kuvauksen muokkaus. Poisto ja käytöstä poisto kuuluvat tärkeään jatkoon (P1): tee ne vain, jos viikosta jää aikaa.
- ☐ Tarkista kuormitus. Jos viikko uhkaa paisua, pidä ydin eli projektit, tehtävälajit, jäsenyys ja roolirajaus. Yksinkertaista projektityypit muokattavaksi listaksi samaan näkymään ja kirjaa rajaus.

Valmis kun: Projektipäällikkö voi lisätä tehtävälajin ja projektityypin ja muokata niiden nimeä ja kuvausta, ja mahdollinen kuormitusrajaus on kirjattu.

Tallenna työnäyte: Koodi commitilla. Kuormitusrajauksesta tehdyt päätökset työviikon 6 päiväkirjaan.

Työvaihe 4 / 4 · Toteuta projektin jäsenyydet

- ☐ Toteuta jäsenyys. Toteuta työntekijän lisäys projektiin ja poisto projektista projektin_jasen-tauluun.
- ☐ Rajaa projektivalikko. Rajaa kirjauslomakkeen projektivalikko jäsenyyden mukaan palvelimella, ei vain selaimessa.
- ☐ Testaa hallintareitit. Kokeile jokaista hallintareittiä työntekijän istunnolla ja suoralla URL-osoitteella. Odotettu tulos on 403, ei näkymä. Kirjautumattomana odotettu tulos on 401.

Valmis kun: Työntekijän kirjauslomakkeessa näkyvät vain projektit, joihin hänet on liitetty, ja jokainen hallintareitti palauttaa työntekijälle 403 myös suoralla URL-osoitteella.

Tallenna työnäyte: Testattu reittikartta ja demo molemmilla rooleilla työviikon 6 päiväkirjaan.

Viikon lopputarkistus: Työntekijän kirjauslomakkeessa näkyvät vain projektit, joihin hänet on liitetty; vain projektipäällikkö pääsee hallintanäkymiin (testattu myös suoralla URL-osoitteella); reittikartta on dokumentoitu.

Työnäyte Git-repositoryyn ennen rastia: perusteltu kirjastoalinta package.json-diffillä, reittikartta ja demo molemmilla rooleilla (p7, k4; s7 laajenee).

Työviikko 7 / 18 – Omat kirjaukset haltuun

Yhteys kokonaisprojektiin: Kirjaukset ja jäsenyydet ovat nyt olemassa. Tällä viikolla työntekijä hallitsee omia kirjauksiaan turvallisesti, ja sinä alat kirjata omat projektituntisi TuntiTutkaan. Oma käyttö tuottaa työviikkojen 8 ja 9 raportteihin aitoa dataa ja paljastaa käytettävyyssongelmat ennen asiakasta.

Viikon tavoite: Työntekijä muokkaa ja poistaa vain omia kirjauksiaan, suodattaa niitä viikon ja projektin mukaan ja näkee oman viikkosummansa.

Työvaihe 1 / 4 · Toteuta kirjauksen muokkaus ja poisto

- ☐ Tee muokkaus ja poisto. Lisää omien kirjausten listaan muokkaus ja poisto. Käytä samaa validointia kuin uudessa kirjauksessa.
- ☐ Tarkista omistajuus. Tarkista palvelimella reiteissä PUT ja DELETE /api/kirjaukset/:id, että kirjaus kuuluu kirjautuneelle käyttäjälle. Vieras kirjaus palauttaa 403.
- ☐ Testaa suoralla kutsulla. Yritä muokata toisen käyttäjän kirjausta curlilla toisen käyttäjän istunnolla. Kirjaa odotettu ja saatu vastauskoodi.

Valmis kun: Toisen käyttäjän kirjausta ei voi muokata eikä poistaa edes suoralla rajapintakutsulla: odotettu 403, saatu 403.

Tallenna työnäyte: Omistajuustestin tulos työviikon 7 päiväkirjaan. Koodi commitilla.

Työvaihe 2 / 4 · Lisää suodatus ja päätä viikkokäytäntö

- ☐ Päätä viikkokäytäntö. Valitse ISO-viikko eli kansainvälisen standardin viikkonumerointi, jossa viikko alkaa maanantaista, tai muu tapa. Päätä, miten viikon- ja vuodenvaihde käsitellään.
- ☐ Kirjaa päätös. Kirjoita valinta perusteluineen suunnitelman Viikkokäytäntö-kenttään, ennen kuin kirjoitat kyselyn.
- ☐ Toteuta suodattimet. Lisää omien kirjausten listaan viikko- ja projektisuodatus reitille GET /api/kirjaukset/omat?viikko=&projekti=.

Valmis kun: Lista näyttää valitun viikon ja projektin kirjaukset, ja viikkokäytäntö on kirjattu suunnitelmaan.

Tallenna työnäyte: Valittu viikkokäytäntö perusteluineen työviikon 7 päiväkirjaan. Koodi commitilla.

Työvaihe 3 / 4 · Laske oma viikkosumma ja päivitä yhteystiedot

- ☐ Laske viikkosumma. Laske oma viikkosumma kyselyllä reitillä GET /api/kirjaukset/omat/viikkosumma?viikko=. Älä tallenna summaa mihinkään.
- ☐ Vertaa käsin laskettuun. Laske saman viikon tunnit käsin ja vertaa niitä sovelluksen summaan.
- ☐ Toteuta yhteystiedot. Toteuta omien yhteystietojen eli puhelimen ja sähköpostin päivitys.

Valmis kun: Viikkosumma täsmää käsin laskettuun eikä sitä ole tallennettu tietokantaan, ja käyttäjä voi päivittää omat yhteystietonsa.

Tallenna työnäyte: Käsin lasketun ja sovelluksen summan vertailu työviikon 7 päiväkirjaan.

Työvaihe 4 / 4 · Aloita omien projektituntien kirjaaminen

- ☐ Lisää tehtävlajit. Lisää TuntiTutkaan tehtävlajit suunnittelu, koodaus, testaus ja dokumentointi.
- ☐ Kirjaa tuntisi. Kirjaa tästä eteenpäin omat projektityötuntisi TuntiTutkaan tehtävlajeittain.
- ☐ Vertaa arvioihin. Vertaa toteutuneita tunteja issueiden työmääräarvioihin ja aloita arvio vs. toteuma -taulukko.

Valmis kun: Sovelluksessa on vähintään viikon verran omia kirjauksiasi, ja arvio vs. toteuma -taulukossa on ensimmäiset rivit.

Tallenna työnäyte: Kuvakaappaus omista kirjauksista ja ensimmäinen arvioiden ja toteuman vertailu työviikon 7 päiväkirjaan.

Viikon lopputarkistus: Toisen käyttäjän kirjausta ei voi muokata edes suoralla API-kutsulla (testi kirjattu: odotettu 403, saatu 403); viikkosumma täsmää käsin laskettuun; omia kirjauksia on sovelluksessa vähintään viikon verran.

Työnäyte Git-repositoryyn ennen rastia: API-testin tulos (403), viikkosumman käsinlaskuvertailu, kuvakaappaus omista kirjauksista ja arvio vs. toteuma -taulukon alku (p7, s6, s7).

Vaihe 3 – Yhteenvedot ja asiakaskatselmointi

Työviikko 8 / 18 – Yhteenvedojen laskenta

Yhteys kokonaisprojektiin: Kirjauksia on nyt kertynyt myös omasta käytöstäsi. Tällä viikolla lasket niistä sen tiedon, jota asiakas on koko ajan halunnut: tunnit viikoittain, henkilöittäin ja tehtävälajeittain. Työviikolla 9 samat luvut saavat kaaviot, ja työviikolla 10 asiakkaan roolissa oleva testaaja kokeilee niitä.

Viikon tavoite: Projektipäällikkö näkee ajantasaisen taulukkoyhteenvedon kolmella ryhmittelyllä ja pääsee rivistä yksittäisiin kirjauksiin.

Työvaihe 1 / 4 · Suunnittele raporttien vastausmuoto

- ☐ Piirrä vastaukset. Piirrä kolmen raporttireitin JSON-vastaukset paperille: mitkä kentät ja missä muodossa.
- ☐ Päätä tyhjä tulos. Päätä, mitä reitti palauttaa, kun aikavälillä ei ole kirjauksia. Tyhjä tulos on normaali tilanne, ei virhe.
- ☐ Nimeä reitit. Kirjaa reitit ja parametrit, esimerkiksi `GET /api/raportit/viikot?alkaa=&paattyy=&projekti=` sekä vastaavat `/henkilot` ja `/tehtavalajit`.

Valmis kun: Kolmen raporttireitin vastausmuoto ja tyhjän tuloksen muoto on kirjattu ennen ensimmäistäkään kyselyä.

Tallenna työnäyte: Vastausmuoto kuvana tai tekstinä `project-docs/-` kansioon ja työviikon 8 päiväkirjaan.

Työvaihe 2 / 4 · Rakenna testiaineistot ja laske odotusarvot

- ☐ Rakenna aineistot. Rakenna kolme testiaineistoa: normaali (esimerkiksi 3 käyttäjää × 2 projektia × 3 tehtävälajia), tyhjä viikko ja viikonvaihteen ylittävä aineisto.
- ☐ Laske odotusarvot. Laske jokaiselle aineistolle ja ryhmittelylle odotetut summat käsin.
- ☐ Kirjaa taulukkoon. Kirjaa odotusarvot taulukkoon: aineisto, ryhmittely ja käsin laskettu summa. Sarakkeet rajapinnan palauttamalle summalle ja täsmävyydelle jäävät vielä tyhjiksi.

Valmis kun: Odotusarvotaulukossa on käsin lasketut summat kolmelle aineistolle ja kolmelle ryhmittelylle, ja se on kirjattu ennen ajoa.

Tallenna työnäyte: Odotusarvotaulukko `project-docs/-` kansioon ennen ensimmäistä ajoa.

Työvaihe 3 / 4 · Toteuta laskentamoduuli ja raporttireitit

- ☐ Kirjoita kyselyt. SQL on tietokannan kyselykieli: `GROUP BY` ryhmittelee rivit ja `SUM` laskee ryhmän summan. Toteuta kolme ryhmittelyä kyselyinä (tai perustellusti JavaScriptillä) kansioon `server/laskenta/`.
- ☐ Lisää rajaukset. Lisää aikaväli- ja projektirajaus. Tarkista, että rajaukset koskevat kaikkia kolmea ryhmittelyä samalla tavalla.
- ☐ Tee reitit. Tee kolme raporttireittiä tiedostoon `server/routes/raportit.js`. Reitti vain ottaa parametrit ja kutsuu laskentamoduulia.
- ☐ Kirjoita yksikkötestit. Kirjoita laskentamoduulille vähintään kolme yksikkötestiä: normaali aineisto, tyhjä viikko ja viikonvaihteen ylitys.
- ☐ Vertaa odotusarvoihin. Aja reitit kolmella testiaineistolla ja kirjaa palautetut summat odotusarvotaulukkoon. Tarkista skeemasta, ettei summataulua ole syntynyt.

Valmis kun: Kolme raporttireittiä palauttaa summat, jotka täsmäyvät käsin laskettuihin kolmella aineistolla, yksikkötestit menevät läpi eikä skeemassa ole summataulua.

Tallenna työnäyte: Laskentamoduulin ja testien commit. Odotusarvot ja rajapinnan palauttavat luvut rinnakkain työviikon 8 päiväkirjaan.

Työvaihe 4 / 4 · Rakenna taulukkoyhteenvedo ja porautuminen

- ☐ Rakenna taulukko. Toteuta projektipäällikön taulukkoyhteenvedo: ryhmittely, rivit ja summat selkeästi luettavina.
- ☐ Lisää porautuminen. Toteuta porautuminen: taulukon rivistä pääsee niihin yksittäisiin kirjauksiin, joista summa syntyy.
- ☐ Tarkista summa riveistä. Laske porautumisnäkyvän kirjausten tunnit yhteen ja vertaa taulukon riviin. Lukujen pitää olla samat.

Valmis kun: Projektipäällikkö näkee taulukon kolmella ryhmittelyllä, ja jokaisesta rivistä pääsee niihin kirjauksiin, joista summa syntyy.

Tallenna työnäyte: Kuvakaappaus taulukosta ja porautumisesta työviikon 8 päiväkirjaan. Koodi commitilla.

Viikon lopputarkistus: Kolme raporttireittiä palauttaa summat, jotka täsmäävät käsin laskettuihin odotusarvoihin vähintään kolmella eri testiaineistolla; tietokannassa ei ole summataulua ja skeema todistaa sen; laskentamoduulilla on vähintään kolme yksikkötestiä.

Työnäyte Git-repositoryyn ennen rastia: odotusarvotaulukko (kirjattu ennen ajoa), testiajon tulokset, laskentamoduulin commit ja skeeman kuvaus (s7 päätyönäyte, p4; s10 pohjustuu).

Työviikko 9 / 18 – Raporttinäkymä ja kaaviot

Yhteys kokonaisprojektiin: Työviikon 8 raporttireitit palauttavat oikeat summat, mutta asiakas haluaa nähdä ne yhdellä silmäyksellä. Nyt muunnat saman datan kaavioiksi ja käsittelet tyhjän datan, verkkovirheen ja lataustilan. Näin väliversio on valmis työviikon 10 asiakaskatselmointiin.

Viikon tavoite: Projektipäällikkö näkee viikkotunnit ja tehtävälajijakauman kaavioina, jotka päivittyvät valinnoista eivätkä kaadu tyhjään dataan tai verkkovirheeseen.

Työvaihe 1 / 4 · Valitse kaaviokirjasto ja piirrä ensimmäinen kaavio

- ☐ Vertaa kirjastoja. Vertaa kahta kaaviokirjastoa: koko, lisenssi, dokumentaatio ja saavutettavuus.
- ☐ Perustele ja asenna. Kirjaa valinta perusteluineen suunnitelman Kaaviokirjasto-kenttään ja asenna valittu kirjasto.
- ☐ Piirrä kiinteällä datalla. Piirrä ensimmäinen kaavio kiinteällä esimerkkidatalla, jotta kirjaston käyttö ja oma data eivät sekoitu toisiinsa.

Valmis kun: Kirjastovalinta on perusteltu suunnitelmassa, ja ensimmäinen kaavio piirtyy kiinteällä datalla.

Tallenna työnäyte: Kirjastovertailu project-docs/-kansioon, asennus commitilla.

Työvaihe 2 / 4 · Hae raportit ja kirjoita muunnosfunktio

- ☐ Hae raportit. Hae raporttireittien data raporttinäkymään selaimen fetch-funktiolla.
- ☐ Kirjoita muunnos. Kirjoita funktio, esimerkiksi muunnaKaavioksi tiedostoon client/src/raportit/muunna.js, joka muuntaa rajapinnan JSON-vastauksen kaavion tarvitsemaan muotoon.
- ☐ Testaa muunnos. Kirjoita funktiolle testi, ennen kuin kytket kaavion oikeaan dataan. Aja testi ja kirjaa tulos.
- ☐ Kytke oikea data. Korvaa kiinteä esimerkkidata muunnetulla raporttidatalla.

Valmis kun: Muunnosfunktion testi menee läpi, ja kaaviot piirtyvät oikeasta raporttidatasta.

Tallenna työnäyte: Muunnosfunktio ja testi commitilla. Testin ajotulos työviikon 9 päiväkirjaan.

Työvaihe 3 / 4 · Lisää ryhmittelyn ja aikavälin valinta

- ☐ Lisää ryhmittely. Toteuta ryhmittelyn vaihto: viikko, henkilö ja tehtävälaji.
- ☐ Lisää aikaväli. Toteuta aikavälin valinta, joka välittyy raporttireitin parametreiksi.
- ☐ Tarkista päivitys. Vaihda ryhmittelyä ja aikaväliä ja tarkista, että kaavio päivittyy jokaisesta valinnasta.

Valmis kun: Kaavio päivittyy jokaisesta ryhmittelyn ja aikavälin valinnasta.

Tallenna työnäyte: Kuvakaappaukset kaavioista kahdella eri valinnalla työviikon 9 päiväkirjaan.

Työvaihe 4 / 4 · Käsittele virhetilat ja valmistelee demodata

- ☐ Käsittele tyhjä aineisto. Näytä ohjaava viesti, kun valitulla aikavälillä ei ole kirjauksia. Tyhjä aineisto on normaali tilanne, ei virhe.
- ☐ Käsittele verkkovirhe. Näytä virheilmoitus ja uudelleenyritys. Testaa sammuttamalla backend kesken raporttihaun.
- ☐ Näytä lataustila. Näytä latausindikaattori, kun haku on kesken.
- ☐ Kirjaa virhetilat. Kirjaa taulukkoon jokaisesta tilasta, mitä käyttäjä näkee ja miten se testattiin.
- ☐ Valmistelee demodata. Valmistelee työviikon 10 katselmointia varten testitunnukset molemmille rooleille ja niin paljon kirjauksia, että kaaviot näyttävät järkevältä.

Valmis kun: Tyhjä aineisto, verkkovirhe ja lataus näkyvät käyttäjälle viestinä eivätkä tyhjänä ruutuna, ja demodata on valmiina.

Tallenna työnäyte: Virhetilataulukko ja kuvakaappaukset virhetiloista työviikon 9 päiväkirjaan.

Viikon lopputarkistus: Kaaviot piirtyvät oikeasta datasta ja päivittyvät suodattimista; verkkovirhe (testataan katkaisemalla backend) näyttää viestin eikä tyhjää ruutua; muunnosfunktion testi menee läpi; valinta on perusteltu kirjallisesti.

Työnäyte Git-repositoryyn ennen rastia: kirjastovertailu, muunnosfunktion testi sekä kuvakaappaukset kaavioista ja virhetiloista (s10, k4, k3).

Työviikko 10 / 18 – Asiakaskatselmointi

Yhteys kokonaisprojektiin: Kirjaus, hallinta ja yhteenvedot toimivat julkaistussa versiossa. Nyt asiakkaan roolissa oleva ulkopuolinen testaaja kokeilee sitä omin käsin, ja selviää, ratkaiseeko sovellus asiakkaan ongelman. Palaute puolivälissä ehtii vielä muuttaa suuntaa, ja tärkein muutos toteutetaan työviikolla 11.

Viikon tavoite: Asiakkaan roolissa oleva testaaja on kokeillut julkaistua versiota molemmilla rooleilla, ja sovitut muutokset ovat priorisoituina issueina.

[Lataa dokumentointipohjat \(katselmointiloki\).](#)

Työvaihe 1 / 4 · Valmistele katselmointi

- ☐ Varmista testaaja. Varmista ohjaajan kanssa testaaja ja ajankohta. Asiakkaan roolia ei esitä oma ohjaava opettajasi.
- ☐ Tee esityslista. Kirjoita esityslista ja kaksi testipolkua: työntekijä kirjaa tunnin, projektipäällikkö katsoo raportin ja porautuu.
- ☐ Tarkista tunnukset ja data. Tarkista, että testitunnukset toimivat julkaistussa versiossa ja kirjauksia on tarpeeksi.
- ☐ Kirjaa testattu versio. Kirjaa muistion alkuun testattava versio (commit tai tagi) ja testaajan rooli.

Valmis kun: Testaaja ja ajankohta on sovittu, testipolut on kirjoitettu ja tunnukset toimivat julkaistussa versiossa.

Tallenna työnäyte: Esityslista ja testipolut muistioon `project-docs/katselmointi-vk10.md`.

Työvaihe 2 / 4 · Esittele versio ja anna testaajan kokeilla

- ☐ Esittele asiakaslähtöisesti. Kerro, mitä sovellus tekee työntekijälle ja projektipäällikölle. Jätä tekninen toteutus pois.
- ☐ Anna kokeilla itse. Anna testaajan tehdä molemmat testipolut ilman opastusta. Älä ota hiirtä käteesi, kun hän epäröi.
- ☐ Merkitse pysähdykset. Merkitse muistiin jokainen kohta, jossa testaaja pysähtyi tai kysyi jotain.

Valmis kun: Testaaja on tehnyt molemmat testipolut itse, ja pysähdyskohdat on merkitty.

Tallenna työnäyte: Pysähdyskohdat katselmointimuistioon jo katselmoinnin aikana.

Työvaihe 3 / 4 · Kirjaa palaute sitaatteina ja oma tulkinta erikseen

- ☐ Kirjaa sitaatit. Kirjoita testaajan havainnot heti hänen omin sanoinaan. Älä siisti äläkä tulkitse vielä.
- ☐ Kirjaa tulkinta. Kirjaa erilliseen sarakkeeseen, mitä uskot havainnon tarkoittavan ja mikä sen syy on.
- ☐ Täydennä muistio. Täydennä muistioon testaajan rooli, ajankohta ja testattu versio.

Valmis kun: Muistiossa jokaisella havainnolla on testaajan omat sanat ja erillinen oma tulkinta.

Tallenna työnäyte: `project-docs/katselmointi-vk10.md`. Kolme tärkeintä sitaattia työviikon 10 päiväkirjaan.

Työvaihe 4 / 4 · Priorisoi muutokset ja kirjaa ne issueiksi

- ☐ Priorisoi ohjaajan kanssa. Käy muutokset läpi ohjaajan kanssa ja päätä, mikä toteutetaan työviikolla 11 ja mikä jää v1.1-listalle.
- ☐ Kirjaa issueiksi. Kirjaa sovitut muutokset GitHub-issueiksi hyväksymiskriteereineen ja arvioineen.
- ☐ Kirjaa päätökset muistioon. Lisää muistioon sovitut muutokset, niiden prioriteetit ja issue-numerot.

Valmis kun: Tärkein muutos on issueina hyväksymiskriteereineen, ja muistiossa näkyvät sovitut muutokset issue-numeroineen.

Tallenna työnäyte: Muistio ja issueet. Ensimmäiseksi valittu muutos perusteluineen työviikon 10 päiväkirjaan.

Viikon lopputarkistus: Katselmointimuistio (testaajan nimetty rooli, ajankohta, testaajan omat sanat, oma tulkinta, sovitut muutokset) on `project-docs`-kansiossa ja tärkein muutos on issueina hyväksymiskriteereineen.

Työnäyte Git-repositoryyn ennen rastia: katselmointimuistio, priorisoitu muutoslista ja uudet issueet (s2, s3, p10; s1 täydentyä).

Vaihe 4 – Viimeistely ja laatu

Työviikko 11 / 18 – Palautemuutos

Yhteys kokonaisprojektiin: Työviikon 10 katselmoinnin tärkein muutos muuttuu nyt koodiksi. Teet sen omassa haarassa ja liität pääversioon pull requestilla, koska ominaisuuden hallittu liittäminen olemassa olevaan versioon on oma osaamisvaatimuksensa. Palaute, joka ei johda muutokseen, on kerätty turhaan.

Viikon tavoite: Testaajan tärkein havainto on korjattu julkaistussa versiossa, ja muutos on liitetty pääversioon pull requestilla.

Työvaihe 1 / 4 · Valitse tärkein muutos ja tarkenna kriteerit

- ☐ Valitse muutos. Valitse katselmoinnin tärkein muutos niistä issueista, jotka sovittiin ohjaajan kanssa työviikolla 10.
- ☐ Tarkenna kriteerit. Kirjoita issueen hyväksymiskriteerit, jotka voi tarkistaa julkaistusta versiosta.
- ☐ Linkitä palautteeseen. Lisää issueen viittaus katselmointimuistion kohtaan ja testaajan sitaattiin.

Valmis kun: Issuessa on hyväksymiskriteerit ja viittaus katselmointimuistion kohtaan, ennen kuin olet kirjoittanut riviäkään koodia.

Tallenna työnäyte: Issue GitHubissa. Valittu muutos ja valinnan syy työviikon 11 päiväkirjaan.

Työvaihe 2 / 4 · Toteuta muutos omassa haarassa

- ☐ Luo haara. Luo ominaisuushaara eli oma kehityslinja päivitetystä main-haarasta. Nimeä se tehtävän mukaan, esimerkiksi feature/viikkosuodatin-ylos.
- ☐ Tee pienet commitit. Tee muutos pienissä osissa. Yksi commit on yksi looginen muutos, ja viesti kertoo, mitä ja miksi.
- ☐ Vie haara GitHubiin. Vie haara etärepositoryyn komennolla `git push`, jotta voit avata siitä pull requestin.

Valmis kun: Muutos on omassa haarassa useana pienenä committina, ja haara näkyy GitHubissa.

Tallenna työnäyte: Haara ja commitit GitHubissa.

Työvaihe 3 / 4 · Avaa pull request ja ratkaise merge-konflikti

- ☐ Avaa pull request. Avaa pull request (PR) eli pyyntö yhdistää haara pääversioon. Kirjoita kuvaus: mitä, miksi ja linkki palautteeseen.
- ☐ Katselmoi oma koodi. Kommentoi pull requestiin omat valintasi perusteluineen ja merkitse kohdat, joita epäilet.
- ☐ Ratkaise konflikti. Merge-konflikti syntyy, kun sama kohta on muuttunut molemmissa haaroissa. Ratkaise se tiedosto kerrallaan ja aja testit ennen commitia.
- ☐ Harjoittele tarvittaessa. Jos konfliktia ei synny luonnostaan, tee ohjattu harjoituskonflikti ja merkitse se selvästi harjoitukseksi.
- ☐ Yhdistä pääversioon. Yhdistä haara pääversioon eli tee merge, kun kuvaus ja itsekatselmointi ovat valmiit.

Valmis kun: Pull requestin kuvaus ja kommentit näyttävät, mitä muutettiin ja miksi, ja konfliktin ratkaisu näkyy historiassa.

Tallenna työnäyte: Pull requestin linkki sekä se, miten konflikti syntyi ja miten ratkaisit sen, työviikon 11 päiväkirjaan.

Työvaihe 4 / 4 · Todenna muutos julkaistussa versiossa

- ☐ Julkaise. Julkaise yhdistetty pääversio samalla julkaisuputkella kuin ennenkin.
- ☐ Todenna palautetta vasten. Avaa julkaistu versio ja tee juuri se asia, josta testaaja huomautti. Sen pitää nyt onnistua ilman epäröintiä.
- ☐ Kuittaa muistio. Kuittaa katselmointimuistion kohta linkillä pull requestiin.

Valmis kun: Muutos toimii julkaistussa versiossa, ja katselmointimuistion kohta on kuitattu linkillä.

Tallenna työnäyte: Ennen/jälkeen-kuvakaappaus työviikon 11 päiväkirjaan.

Viikon lopputarkistus: Pull requestin kuvaus ja keskustelu näyttävät mitä muutettiin ja miksi; muutos on tuotannossa; konfliktin ratkaisu näkyy historiassa; katselmointimuistion kohta on kuitattu linkillä.

Työnäyte Git-repositoryyn ennen rastia: pull request -linkki (PR), merge-commit, ennen/jälkeen-kuvakaappaus ja kuitattu muistio (s13; s12 syvenee, p10 jatkuu).

Työviikko 12 / 18 – Mobiili ja saavutettavuus

Yhteys kokonaisprojektiin: Toiminnot ovat kasassa, ja palautemuutos on tuotannossa. Nyt varmistat, että ne toimivat siellä, missä asiakkaan väki niitä käyttää: puhelimella asiakaskäynnillä, näppäimistöllä ja ruudunlukijalla. Mittaus ennen ja jälkeen

tekee parannuksesta todistettavan.

Viikon tavoite: Työntekijä kirjaa tunnin oikealla puhelimella alle 30 sekunnissa, ja koko sovellus toimii pelkällä näppäimistöllä.

Työvaihe 1 / 4 • Mittaa lähtötaso Lighthouseella

- ☐ Aja Lighthouse. Aja Lighthouse eli Chromen kehittäjätyökalujen laatumittari julkaistulle versiolle: saavutettavuus ja suorituskky.
- ☐ Tallenna raportti. Tallenna raportti repositoryyn, ennen kuin korjaat mitään.
- ☐ Kirjaa luvut. Kirjaa saavutettavuuden ja suorituskvyn pistemäärät muistiin.

Valmis kun: Lähtötason raportti on repositoryssa, ja pistemäärät on kirjattu ennen ensimmäistään korjausta.

Tallenna työnäyte: Lighthouse-raportti project-docs/-kansioon, pistemäärät työviikon 12 päiväkirjaan.

Työvaihe 2 / 4 • Korjaa responsiivisuus ja testaa oikealla puhelimella

- ☐ Valitse taitekohdat. Valitse taitekohdat oman sisältösi mukaan: siitä leveydestä, jossa lomake tai taulukko alkaa ahtautua.
- ☐ Korjaa CSS. Korjaa responsiivisuus omilla CSS-taitekohdilla niin, että kirjauslomake ja raportit mahtuvat puhelimen näytölle.
- ☐ Testaa puhelimella. Tee kirjaus oikealla puhelimella alusta loppuun ja ota aika. Tavoite on alle 30 sekuntia.

Valmis kun: Kirjaus on tehty oikealla puhelimella alusta loppuun alle 30 sekunnissa.

Tallenna työnäyte: Kuvakaappaus puhelimelta ja mitattu kirjausaika työviikon 12 päiväkirjaan.

Työvaihe 3 / 4 • Korjaa näppäinpolku ja saavutettavuus

- ☐ Kulje näppäinpolku. Kulje koko sovellus pelkällä näppäimistöllä: kirjautuminen, kirjaus, lista ja raportit. Merkitse jokainen kohta, jossa fokus katoaa tai jumittuu.
- ☐ Kytke labelit. Kytke jokainen lomakekenttä label-elementtiin ja virheviesti ohjelmallisesti kenttäänsä.
- ☐ Korjaa hierarkia ja kontrastit. Korjaa otsikkohierarkia ilman hyppyjä ja varmista kontrastit myös napeissa. CSS-ratkaisut tehdään itse.
- ☐ Lisää kaavion tekstivastine. Anna jokaiselle kaaviolle tekstivastine: sama data taulukkona, jotta tieto on saatavilla ilman kuvaa.
- ☐ Kirjaa korjauslista. Kirjaa jokainen korjaus omalle rivilleen korjauslistaan.

Valmis kun: Koko käyttöpolku kulkee pelkällä näppäimistöllä, jokaisella kentällä on label ja jokaisella kaaviolla tekstivastine.

Tallenna työnäyte: Korjauslista ja näppäinpolun kuvaus työviikon 12 päiväkirjaan.

Työvaihe 4 / 4 • Mittaa uudelleen ja vertaa lähtötasoon

- ☐ Aja Lighthouse uudelleen. Aja sama mittaus samalle sivulle ja tallenna raportti repositoryyn.
- ☐ Kirjoita vertailu. Kirjoita ennen/jälkeen-vertailu: pistemäärät ja ne korjaukset, jotka nostivat niitä.
- ☐ Tarkista tavoite. Tarkista, että saavutettavuuspistemäärä on vähintään 90. Jos ei ole, palaa korjauslistaan.

Valmis kun: Saavutettavuuspistemäärä on vähintään 90, ja molemmat raportit ja vertailu ovat repositoryssa.

Tallenna työnäyte: Loppuraportti ja vertailu project-docs/-kansioon. Lähtö- ja lopputulokset numeroina työviikon 12 päiväkirjaan.

Viikon lopputarkistus: Kirjaus on tehty oikealla puhelimella alusta loppuun alle 30 sekunnissa; koko käyttöpolku kulkee pelkällä näppäimistöllä; saavutettavuuspistemäärä on vähintään 90 ja molemmat raportit ovat repossa.

Työnäyte Git-repositoryyn ennen rastia: Lighthouse-raportit ennen ja jälkeen, kuvakaappaus puhelimelta, korjauslista ja näppäinpolun kuvaus (p6 täydenty, p3 osa).

Työviikko 13 / 18 – Tietoturva

Yhteys kokonaisprojektiin: Kaikki reitit ovat nyt olemassa, joten ne voi käydä läpi kerralla: jokainen reitti jokaisella roolilla. Työaikatiedot ovat henkilöön sidottua tietoa, ja yksi vuotava raporttireitti vie luottamuksen koko järjestelmään. Löydökset korjataan nyt, jotta työviikon 14 testaus tehdään turvallista versiota vastaan.

Viikon tavoite: Jokainen rajapinnan reitti vastaa jokaiselle roolille odotetulla koodilla, ja selitekenttään kirjoitettu skripti näkyy pelkkänä tekstinä.

Työvaihe 1 / 4 · Kartoita riskit ja kirjaa arvio

- ☐ Käy riskiluokat läpi. Käy läpi viisi riskiluokkaa: syötteet, käyttöoikeudet, salasanat, istunnot ja tietojen näkyvyys.
- ☐ Kirjaa arvio. Kirjaa jokaisesta riskiluokasta riski, ratkaisu ja jännösriski.
- ☐ Tarkista kyselyt. Tarkista, että SQL-kyselyt käyttävät parametreja eivätkä yhdistä käyttäjän syötettä merkkijonoon.

Valmis kun: Riskiarviossa on viisi riskiluokkaa, ja jokaisella on riski, ratkaisu ja jännösriski.

Tallenna työnäyte: Arvio project-docs/-kansioon. Riskiluokat ja ratkaisut työviikon 13 päiväkirjaan.

Työvaihe 2 / 4 · Testaa jokainen reitti jokaisella roolilla

- ☐ Tee reittitaulukko. Tee taulukko, jossa on jokainen rajapinnan reitti kertaa kolme roolia: kirjautumaton, työntekijä ja projektipäällikkö.
- ☐ Kirjaa odotukset. Kirjaa jokaiselle riville odotettu vastauskoodi ennen ajoa.
- ☐ Aja taulukko. Aja jokainen rivi kolmella istunnolla ja kirjaa saatu vastauskoodi odotetun viereen.

Valmis kun: Jokaisella reittitaulukon rivillä on odotettu ja saatu vastaus, ja poikkeamat on merkitty.

Tallenna työnäyte: Ajettu reittitaulukko project-docs/-kansioon.

Työvaihe 3 / 4 · Korjaa löydökset ja siirrä salaisuudet .env-tiedostoon

- ☐ Korjaa poikkeamat. Korjaa jokainen reittitaulukon poikkeama ja aja rivi uudelleen.
- ☐ Siirrä salaisuudet. Siirrä salaisuudet .env-tiedostoon ja varmista, että se on .gitignore-tiedostossa.
- ☐ Tarkista Git-historia. Tarkista Git-historia salaisuuksien varalta, ei vain nykyistä tilaa.

Valmis kun: Jokainen reittitaulukon rivi täsmää odotettuun uusintatestin jälkeen, eikä repositoryssa tai sen historiassa ole salaisuuksia.

Tallenna työnäyte: Korjausten ja .env-järjestelyn commitit. Poikkeamat ja korjaukset työviikon 13 päiväkirjaan.

Työvaihe 4 / 4 · Testaa XSS ja kokoa tietoturva-arvio

- ☐ Syötä skripti. XSS tarkoittaa hyökkäystä, jossa syötekenttään kirjoitettu skripti ajetaan toisen käyttäjän selaimessa. Kirjaa selitteeksi `<script>alert(1)</script>`.
- ☐ Tarkista näkymät. Avaa jokainen näkymä, jossa selite näkyy. Syötteen pitää näkyä tekstinä, eikä konsolissa saa olla virhettä.
- ☐ Kokoa arvio. Kokoa riskiarvio, reittitaulukko, korjaukset ja XSS-testin tulos yhdeksi dokumentiksi project-docs/-kansioon.

Valmis kun: Skriptisyöte näkyy tekstinä kaikissa näkymissä, ja tietoturva-arvio on yhtenä dokumenttina repositoryssa.

Tallenna työnäyte: XSS-testin kuvakaappaus ja tietoturva-arvio project-docs/-kansioon. Jännösriski, jota et voinut poistaa, työviikon 13 päiväkirjaan.

Viikon lopputarkistus: Reittitaulukko on ajettu ja jokainen rivi täsmää odotettuun (poikkeamat korjattu ja uusintatestattu); skripti-syöte renderöityy tekstinä; repossa tai sen historiassa ei ole salaisuuksia; arvioidokumentti on project-docs-kansiossa.

Työnäyte Git-repositoryyn ennen rastia: tietoturva-arvio, ajettu reittitaulukko, XSS-testin kuvakaappaus ja .env-järjestelyn commit (s11; s7 täydentyy).

Työviikko 14 / 18 – Testausviikko

Yhteys kokonaisprojektiin: Tietoturvakorjausten jälkeen koko sovellus testataan kerralla, koska nyt on olemassa kaikki se, mitä testimatriisi on työviikolta 2 asti odottanut. Raportointisovellus, jonka summiin ei voi luottaa, on hyödytön. Löydetyt virheet korjataan täydellisinä ketjuina, ja regressiotestit suojaavat laskentaa julkaisuun asti.

Viikon tavoite: Vähintään 12 suunniteltua testitapausta on ajettu odotusarvoja vasten, ja vähintään kaksi virhettä on korjattu täydellisinä virheenkorjausketjuina.

[Lataa dokumentointipohjat \(testimatriisi\).](#)

Työvaihe 1 / 4 · Täydennä testimatriisi ja kirjaa odotukset

- ☐ Täydennä tapaukset. Täydennä testimatriisi vähintään 12 tapaukseen. Jokainen testitapaus saa tunnuksen T01–T13, jotta siihen voi viitata päiväkirjassa ja korjausketjussa.
- ☐ Jaa luokkiin. Jaa tapaukset kolmeen luokkaan: normaali käyttö, rajat ja virhetilanteet.
- ☐ Kirjaa odotukset. Kirjoita jokaiselle tapaukselle odotettu tulos, ennen kuin ajat sen.

Valmis kun: Matriisissa on vähintään 12 tapausta kolmessa luokassa, ja jokaisella on odotettu tulos ennen ajoa.

Tallenna työnäyte: Testimatriisi project-docs/-kansioon tai dokumentointipohjan testimatriisiin.

Työvaihe 2 / 4 · Aja testit ja kirjaa havainnot

- ☐ Aja tapaukset. Aja jokainen testitapaus ja kirjaa havaittu tulos omaan sarakkeeseensa odotetun viereen, myös silloin, kun se on sama kuin odotus.
- ☐ Merkitse ajankohta. Merkitse jokaiseen tapaukseen ajankohta ja tulos: läpi tai poikkeama.
- ☐ Laske tulos. Laske, montako tapausta meni läpi ensimmäisellä ajolla.

Valmis kun: Testiraportissa jokaisella tapauksella on luokka, odotettu ja havaittu tulos sekä ajankohta.

Tallenna työnäyte: Testiraportti project-docs/-kansioon. Ensimmäisellä ajolla läpi menneiden määrä työviikon 14 päiväkirjaan.

Työvaihe 3 / 4 · Korjaa virheet virheenkorjausketjuina

- ☐ Selvitä syy. Selvitä jokaisen poikkeaman syy selaimen kehittäjätyökaluilla, lokeilla tai testiaineistolla. Arvaus ei riitä.
- ☐ Korjaa ketjuna. Korjaa virhe ketjuna: havainto → toistamisohje → syy → korjauscommit → uusintatesti → regressiotesti.
- ☐ Kirjaa kaksi ketjua. Kirjaa vähintään kaksi ketjua kokonaisina, kaikki kuusi kohtaa commit-viittauksineen.
- ☐ Tarkista aitous. Älä keksi virheitä. Jos aitoja ei löydy, pyydä ohjaajaa merkitsemään vikatehtäviä, joista ketjut ajetaan.

Valmis kun: Vähintään kaksi virheenkorjausketjua on täydellisiä commit-viittauksineen ja perustuu aitoihin havaintoihin.

Tallenna työnäyte: Ketjut project-docs/-kansioon tai dokumentointipohjaan. Kaksi ketjua kokonaisina työviikon 14 päiväkirjaan.

Työvaihe 4 / 4 · Lisää laskennan regressiotestit

- ☐ Kirjoita testit. Lisää laskentamoduulille yksikkötestit viikkorajoille, tyhjälle viikolle ja vuodenvaihteelle.
- ☐ Aja komennolla. Aja testit yhdellä komennolla puhtaassa työhakemistossa ilman käsin tehtyjä vaiheita.
- ☐ Tarkista tulos. Tarkista, että kaikki testit menevät läpi, ja kirjaa ajotulos.

Valmis kun: Regressiotestit ajetaan yhdellä komennolla ja menevät läpi.

Tallenna työnäyte: Testikoodin commit ja ajotulos työviikon 14 päiväkirjaan.

Viikon lopputarkistus: Testiraportissa jokaisella tapauksella on luokka, odotettu ja havaittu tulos sekä ajankohta; vähintään kaksi ketjua on täydellisiä commit-viittauksineen; regressiotestit ajetaan komennolla ja menevät läpi.

Työnäyte Git-repositoryyn ennen rastia: testiraportti (vähintään 12 riviä), kaksi täydellistä virheenkorjausketjua ja testikoodin commitit (p2, p3; k5 testausosa).

Työviikko 15 / 18 – Laatu ja dokumentointi

Yhteys kokonaisprojektiin: Testit ovat vihreinä, joten koodia uskaltaa nyt siistiä: jos refaktorointi rikkoo jotain, regressiotestit kertovat sen heti. Samalla kirjoitat käyttöönotto-ohjeen ja pikaohjeet, koska niitä testataan oikeasti työviikolla 16. Ilman toimivaa ohjetta julkaisutestaus epäonnistuu.

Viikon tavoite: Uusi käyttäjä pääsee alkuun pelkillä ohjeilla, ja refaktoroitu koodi toimii samoin kuin ennen, mikä näkyy vihreinä testeinä.

Työvaihe 1 / 4 · Refaktoroi kolme kohdetta testit vihreinä

- ☐ Valitse kohteet. Valitse kolme kohdetta: pitkä funktio, toistuva koodi ja epäselvä nimi. Perustele jokainen valinta yhdellä lauseella.
- ☐ Refaktoroi yksi kerrallaan. Refaktoroi yksi kohde kerrallaan ja aja regressiotestit jokaisen jälkeen.
- ☐ Tallenna diffit. Tallenna jokaisesta ennen/jälkeen-diffi ja kirjoita, miksi jälkimmäinen on ylläpidettävämpi.

Valmis kun: Kolme refaktorointia on tehty, regressiotestit menevät läpi jokaisen jälkeen, ja diffit perusteluineen ovat repositoryssa.

Tallenna työnäyte: Ennen/jälkeen-diffit ja perustelut project-docs/-kansioon. Kohteet ja valinnan syyt työviikon 15 päiväkirjaan.

Työvaihe 2 / 4 · Kirjoita README ja käyttöönotto-ohje

- ☐ Täydennä README. Täydennä README: mikä sovellus on ja kenelle, vaatimukset, käynnistys, tietokannan alustus ja testien ajo.
- ☐ Kirjoita käyttöönotto-ohje. Kirjoita project-docs/kayttoonotto.md, jolla sovelluksen saa asennettua tyhjään ympäristöön vaihe vaiheelta.
- ☐ Dokumentoi ympäristömuuttujat. Lisää esimerkkitiedosto .env.esimerkki ja kerro ohjeessa, mihin .env luodaan.
- ☐ Pyydä lukija. Anna ohje toiselle luettavaksi ja pyydä häntä merkitsemään jokainen kohta, jota hän ei ymmärrä ilman lisäkysymyksiä.

Valmis kun: README ja käyttöönotto-ohje ovat repositoryssa, ja ohjetta voi seurata kirjaimellisesti ilman aiempaa tietoa projektista.

Tallenna työnäyte: README.md ja project-docs/kayttoonotto.md commitilla.

Työvaihe 3 / 4 · Kirjoita pikaohjeet ja kokoa riippuvuustaulukko

- ☐ Kirjoita työntekijän ohje. Kirjoita project-docs/pikaohje-tyontekija.md: miten tunnit kirjataan.
- ☐ Kirjoita projektipäällikön ohje. Kirjoita project-docs/pikaohje-projektipaallikko.md: projektit, tehtävälajit ja raportit.
- ☐ Kokoa riippuvuudet. Kokoa taulukko: paketti → rooli tässä projektissa → versio. Jos paketille ei löydy roolia, harkitse sen poistoa.

Valmis kun: Molemmille rooleille on pikaohje käyttäjän kielellä, ja jokaisella riippuvuudella on taulukossa rooli ja versio.

Tallenna työnäyte: Pikaohjeet ja riippuvuustaulukko repositoryyn. Kenelle kirjoitit minkäkin ohjeen, työviikon 15 päiväkirjaan.

Työvaihe 4 / 4 · Selvitä lisenssi ohjaajan kanssa

- ☐ Kysy ohjaajalta. Kysy lisenssiä ohjaajalta, jos sitä ei ole vielä sovittu.
- ☐ Kirjaa tila. Kirjaa vastaus suunnitelman Lisenssi-kenttään. Jos vastausta ei vielä ole, jätä kenttä avoimeksi asiaksi äläkä päätä itse.
- ☐ Tarkista LICENSE. Jos lisenssi on sovittu, lisää LICENSE-tiedosto repositoryyn. Jos ei ole, kirjaa asia päiväkirjaan avoimena.

Valmis kun: Lisenssin tila on kirjattu suunnitelmaan: sovittu lisenssi on LICENSE-tiedostona, tai avoin asia on merkitty avoimeksi.

Tallenna työnäyte: Suunnitelman päivitys commitilla. Lisenssikysymyksen tila työviikon 15 päiväkirjaan.

Viikon lopputarkistus: Regressiotestit menevät läpi refaktoroinnin jälkeen; ennen/jälkeen-diffit ja perustelut ovat repossa; dokumentit ovat repossa ja ohjeet on kirjoitettu käyttäjälle, ei arvioijalle; lisenssin tila on kirjattu.

Työnäyte Git-repositoryyn ennen rastia: ennen/jälkeen-diffit perusteluineen, README, käyttöönotto-ohje, riippuvuustaulukko ja pikaohjeet (p5, k7).

Vaihe 5 – Julkaisu ja näyttö

Työviikko 16 / 18 – Julkaisuehdokas ja julkaisutestaus

Yhteys kokonaisprojektiin: Dokumentaatio on kirjoitettu ja sovellus testattu. Nyt jäädytät sisällön ja panet sekä sovelluksen että ohjeen koetukselle: ensin itse puhtaassa ympäristössä, sitten ulkopuolisen testaajan käsissä. Kun julkaisutestaus tehdään julkaisuehdokasta vasten, työviikko 17 jää kokonaan löydösten korjaamiseen ja v1.0:n julkaisuun.

Viikon tavoite: Ulkopuolinen testaaja on ottanut julkaisuehdokkaan v1.0-rc1 käyttöön pelkän kirjallisen ohjeen avulla ja kokeillut molempia rooleja.

Työvaihe 1 / 4 • Tee jäädytyspäättös ja luokittele issueet

- ☐ Kirjaa jäädytys. Päättä ja kirjaa: uusia ominaisuuksia ei enää lisätä.
- ☐ Luokittele issueet. Käy jäljellä olevat issueet läpi ja merkitse ne estäviksi tai v1.1-listalle.
- ☐ Kirjaa v1.1-lista. Kirjaa, mitä jätit tietoisesti v1.1-listalle.

Valmis kun: Jäädytyspäättös on kirjattu, ja jokainen avoin issue on merkitty estäväksi tai v1.1-listalle.

Tallenna työnäyte: Jäädytyspäättös ja v1.1-lista työviikon 16 päiväkirjaan.

Työvaihe 2 / 4 • Julkaise julkaisuehdokas v1.0-rc1

- ☐ Tee tuotantobuild. Tee tuotantobuild puhtaassa hakemistossa.
- ☐ Tarkista asetukset. Tarkista ympäristöasetukset: portti, tietokantatiedoston polku ja pysyvyys, salaisuudet ja `NODE_ENV`.
- ☐ Julkaise ja tee savutesti. Julkaise ja aja savutesti julkisessa osoitteessa: kirjaus ja raportti. Tarkista, että lokit näkyvät alustan lokinäkymässä.
- ☐ Merkitse tagilla. Merkitse versio Git-tagilla `v1.0-rc1` ja vie tagi etärepositoryyn. Tagi on versiomerkitä, joka kiinnitetään yhteen committiin.

Valmis kun: v1.0-rc1 on julkisessa osoitteessa ja merkitty tagilla, ja savutesti on ajettu.

Tallenna työnäyte: Tagi `v1.0-rc1` ja release GitHubissa. Savutestin tulos työviikon 16 päiväkirjaan.

Työvaihe 3 / 4 • Asenna sovellus itse puhtaaseen ympäristöön

- ☐ Asenna ohjeella. Asenna sovellus puhtaaseen ympäristöön pelkän käyttöönotto-ohjeen avulla.
- ☐ Pidä pöytäkirjaa. Kirjaa tiedostoon `project-docs/asennuspöytäkirja-vk16.md` jokainen kohta, jossa epäroit: mitä tein, mihin pysähdyin ja mitä ohjeeseen lisättiin.
- ☐ Korjaa ohje. Korjaa ohje pöytäkirjan perusteella, ennen kuin annat sen ulkopuoliselle. Tallenna korjausdiffi.

Valmis kun: Asennuspöytäkirja on repositoryssa, ja ohjetta on korjattu sen perusteella ennen ulkopuolisen testausta.

Tallenna työnäyte: Asennuspöytäkirja ja ohjeen korjausdiffi repositoryyn. Epärointikohdat työviikon 16 päiväkirjaan.

Työvaihe 4 / 4 • Järjestä ulkopuolinen julkaisutestaus

- ☐ Anna ohje ja osoite. Anna nimetylle ulkopuoliselle käyttöönotto-ohje, julkinen osoite ja kirjallinen tehtävälista. Älä auta suullisesti.
- ☐ Testauta molemmat roolit. Testaaja pääsee alkuun, kirjaa tunnin työntekijänä ja katsoo raportin projektipäällikkönä. Katso kellosta, kauanko alkuun pääseminen kesti.
- ☐ Kirjaa havainnot. Kirjaa havainnot testaajan omin sanoin tiedostoon `project-docs/julkaisutestaus-vk16.md` ja oma tulkinta erikseen.
- ☐ Kirjaa estävät issueiksi. Luokittele havainnot estäviksi tai v1.1-listalle ja kirjaa estävät issueiksi. Niitä ei korjata kiireellä tällä viikolla vaan työviikolla 17.

Valmis kun: Testauspöytäkirjassa on nimetty rooli, ajankohta, testaajan omat sanat ja oma tulkinta erikseen, ja estävät virheet ovat issueina.

Tallenna työnäyte: Julkaisutestauksen pöytäkirja ja estävien issueiden lista repositoryyn. Tärkeimmät havainnot sitaatteina työviikon 16 päiväkirjaan.

Viikon lopputarkistus: v1.0-rc1 on julkisessa osoitteessa ja merkitty tagilla; oma asennuspöytäkirja ja ulkopuolisen testauspöytäkirja (nimetty rooli, ajankohta, testaajan omat sanat erillään omasta tulkinnasta) ovat repossa; estävät virheet on kirjattu issueiksi (niitä ei korjata kiireellä tällä viikolla vaan seuraavalla).

Työnäyte Git-repositoryyn ennen rastia: julkaisuehdokkaan tagi (v1.0-rc1) ja release, oma asennuspöytäkirja, ohjeen korjausdiffi, ulkopuolisen testauspöytäkirja ja estävien issue-lista (k6, s14 osa, s3 toinen katselmointi).

Työviikko 17 / 18 – Julkaisu v1.0

Yhteys kokonaisprojektiin: Työviikon 16 julkaisutestauksen havainnot muuttuvat nyt korjauksiksi, ja julkaisuehdokkaasta tulee v1.0. Koska testaus tehtiin jo julkaisuehdokasta vasten, tämä viikko riittää estävien virheiden korjaamiseen ilman kiirettä. Loppuviikko on puskuria, jota ei täytetä uusilla ominaisuuksilla.

Viikon tavoite: Asiakas saa v1.0:n julkisesta osoitteesta ja luovutusviestin, jolla tiimi pääsee alkuun.

Työvaihe 1 / 4 · Korjaa estävät virheet virheenkorjausketjuna

- ☐ Korjaa ketjuna. Korjaa työviikon 16 estävät havainnot täydellisenä ketjuna: havainto → toistamisohje → syy → korjauscommit → uusintatesti → regressiotesti.
- ☐ Tarkista lähtöhavainto. Jos estäviä ei löytynyt, aja ketju aiemmasta aidosta julkaisuehdokasvaiheen havainnosta. Keksittyjä virheitä ei kirjata.
- ☐ Korjaa vain estävät. Jätä muut havainnot v1.1-listalle. Lista on olemassa juuri tätä varten.

Valmis kun: Kolmas virheenkorjausketju on täydellisenä repositoryssa, ja siinä kerrotaan, mistä aidosta havainnosta se ajettiin.

Tallenna työnäyte: Ketju project-docs/-kansioon. Korjatut estävät virheet ja korjaustapa työviikon 17 päiväkirjaan.

Työvaihe 2 / 4 · Julkaise v1.0 tagilla ja releasella

- ☐ Aja regressiotestit. Aja koko regressiotestisarja ennen tagia ja kirjaa tulos.
- ☐ Merkitse tagilla. Merkitse versio Git-tagilla v1.0 ja vie tagi etärepositoryyn.
- ☐ Tee release. Tee GitHubiin release eli julkaisumerkintä, johon liitetään versio ja sen kuvaus.
- ☐ Julkaise tuotantoon. Julkaise v1.0 tuotantoon samalla julkaisuputkella kuin julkaisuehdokas.

Valmis kun: v1.0 on julkisessa osoitteessa, ja se on merkitty tagilla ja releasella regressiotestien jälkeen.

Tallenna työnäyte: Tagi v1.0 ja release GitHubissa. Regressioajon tulos työviikon 17 päiväkirjaan.

Työvaihe 3 / 4 · Aja savutesti ja kirjoita julkaisutiedote

- ☐ Aja savutesti. Aja testipolut julkaistulle v1.0:lle: kirjaus ja raportti molemmilla rooleilla. Kirjaa tulos.
- ☐ Kirjoita tiedote. Kirjoita julkaisutiedote project-docs/-kansioon: mitä sovellus tekee, mitä kumpikin rooli voi tehdä, tunnetut rajoitteet ja v1.1-lista.
- ☐ Tarkista rajoitteet. Lue tunnetut rajoitteet vielä kerran: ne kerrotaan rehellisesti, ei kaunistellen.

Valmis kun: Savutestin tulos ja julkaisutiedote ovat repositoryssa.

Tallenna työnäyte: Savutestin kirjaus ja julkaisutiedote repositoryyn. Tiedotteen ydin työviikon 17 päiväkirjaan.

Työvaihe 4 / 4 · Kirjoita luovutusviesti asiakkaalle

- ☐ Kirjoita viesti. Kirjoita luovutusviesti asiakaskielellä: osoite, tunnukset, miten pääsee alkuun ja mistä pikaohjeet löytyvät molemmille rooleille.
- ☐ Kerro rajat. Kerro, mitä sovellus ei tee ja keneen otetaan yhteyttä, jos jokin ei toimi.
- ☐ Poista jargon. Lue viesti ääneen ja poista jokainen sana, jota asiakas ei käyttäisi itse.
- ☐ Käytä puskuri tarkistuksiin. Käytä loppuviikko tarkistuksiin. Älä aloita mitään uutta ominaisuutta.

Valmis kun: Luovutusviesti on repositoryssa ilman teknistä jargonia, ja viikolle jäi puskuriaikaa eikä mitään uutta aloitettu.

Tallenna työnäyte: Luovutusviesti project-docs/-kansioon. Se, mitä jätit tietoisesti v1.1-listalle, työviikon 17 päiväkirjaan.

Viikon lopputarkistus: v1.0 on julkisessa osoitteessa ja merkitty tagilla; kolmas ketju on täydellisenä repossa (tai kirjaus siitä, mistä aidosta havainnosta ketju ajettiin); savutestin tulos, julkaisutiedote ja luovutusviesti ovat repossa; viikolle jäi puskuriaikaa eikä mitään uutta aloitettu.

Työnäyte Git-repositoryyn ennen rastia: kolmas virheenkorjausketju, v1.0-tag ja release, savutestin kirjaus, julkaisutiedote ja luovutusviesti (s14, s2; k6 ja p2 täydentyvät).

Työviikko 18 / 18 – Näyttö

Yhteys kokonaisprojektiin: Sovellus on luovutettu, eikä mitään uutta enää rakenneta. Viimeinen viikko kokoa 17 työviikon aineiston niin, että arvioija löytää jokaisen työnäytteen, koska osaaminen, jota arvioija ei löydä, ei ole hänelle olemassa. Demo ja itsearviointi näyttävät, mitä osaat itse perustella.

Viikon tavoite: Arvioija löytää näyttömatriisin linkeistä työnäytteen jokaiseen 32 vaatimukseen, ja 8–10 minuutin demo on harjoiteltu.

[Avaa näyttömatriisi](#)

Työvaihe 1 / 4 • Täsmälinkitä näyttömatriisi

- ☐ Käy vaatimukset läpi. Käy kaikki 32 vaatimusta läpi ja liitä jokaiseen viikko, työnäyte ja toimiva linkki: commit, issue, dokumentti tai kuvakaappaus.
- ☐ Etsi aukot. Etsi kohdat, joissa linkki puuttuu tai osoittaa epämääräiseen aineistoon.
- ☐ Korjaa aukot. Korjaa heikoimmin todennetut kohdat nyt, kun aikaa vielä on.
- ☐ Tarkista aineisto. Tarkista, että päiväkirja, AI-loki ja testiraportti ovat repositoryssa ja linkit aukeavat.

Valmis kun: Jokainen matriisin rivi osoittaa olemassa olevaan aineistoon, ja jokainen linkki aukeaa.

Tallenna työnäyte: Näyttömatriisi täsmälinkitettyinä. Heikoimmin todennetut kohdat ja niiden korjaus työviikon 18 päiväkirjaan.

Työvaihe 2 / 4 • Harjoittele demo toiselle henkilölle

- ☐ Tee demorunko. Rakenna runko: kirjaus → raportti ja porautuminen → yksi tekninen ratkaisu (lasketut yhteenvedot) → yksi korjattu virhe ketjuineen → Git-historia → AI-lokin tarkistettu käyttö.
- ☐ Harjoittele kellon kanssa. Esitä demo toiselle henkilölle ja ota aika. Tavoite on 8–10 minuuttia.
- ☐ Korjaa runkoa. Korjaa runkoa sen mukaan, mihin aika meni ja mitä kuulija kysyi.

Valmis kun: Demo on ajettu kellon kanssa vähintään kerran toiselle henkilölle, ja sen kesto on kirjattu.

Tallenna työnäyte: Demorunko repositoryyn. Demon kesto harjoituksessa työviikon 18 päiväkirjaan.

Työvaihe 3 / 4 • Kirjoita itsearviointi

- ☐ Kirjoita onnistumiset. Kirjoita, mikä omassa työskentelyssäsi onnistui. Nimeä tilanne ja työviikko.
- ☐ Kirjoita avun tarve. Kirjoita, missä tarvitsit apua ja keneltä sait sen.
- ☐ Kirjoita, mitä tekisit toisin. Kirjoita, mitä tekisit seuraavassa projektissa toisin ja miksi.

Valmis kun: Itsearviointinissa on konkreettisia tilanteita työviikkoineen, ei yleislauseita.

Tallenna työnäyte: Itsearviointi repositoryyn. Sen ydin työviikon 18 päiväkirjaan.

Työvaihe 4 / 4 • Tarkista aukot ja luovuta näyttöpaketti

- ☐ Käytä puskuripäivä. Käy aineisto läpi toisen henkilön kanssa ja korjaa löydettyt aukot. Mitään uutta ei rakenneta.
- ☐ Kokoa paketti. Kokoa näyttöpaketti: täsmälinkitetty näyttömatriisi, projektipäiväkirja, AI-loki, demorunko, itsearviointi ja julkaistu v1.0.
- ☐ Luovuta. Luovuta paketti arvioijalle ohjaajan kanssa sovitulla tavalla.

Valmis kun: Aukkojen tarkistus toisen henkilön kanssa on tehty, ja näyttöpaketti on luovutettu arvioijalle.

Tallenna työnäyte: Luovutettu paketti. Luovutuksen ajankohta ja sisältö työviikon 18 päiväkirjaan.

Viikon lopputarkistus: Jokainen matriisin rivi osoittaa olemassa olevaan aineistoon ja linkki aukeaa; demo on ajettu kellon kanssa vähintään kerran toiselle henkilölle; itsearviointi sisältää konkreettisia tilanteita, ei yleislauseita.

Työnäyte Git-repositoryyn ennen rastia: valmis näyttömatriisi, demorunko, itsearviointi ja luovutettu paketti (p11 ja koko matriisin koonti).

Viimeiset viisi päivää

Ma · Sisältöjäädytys: viimeinen hyväksytty versio, matriisin täsmälinkitys alkaa

Ti · Aineisto: päiväkirja, AI-loki, testiraportti ja linkkien tarkistus

Ke · Demoharjoitus kellon kanssa (8–10 min) ja itsearvioinnin kirjoittaminen

To · Puskuri: aukkojen korjaus ja tarkistus toisen henkilön kanssa

Pe · Luovutus: näyttömatriisi, projektipäiväkirja, AI-loki ja julkaistu v1.0

Sanasto

Projektin tunnukset ja ammattitermit siinä järjestyksessä, jossa ne tulevat vastaan. Jokainen on selitetty myös sivustolla ensimmäisen käytön kohdalla.

repository vko 1	<i>repo, projektin kansio versionhallinnassa.</i> GitHubissa oleva projektin kansio, jossa koodi, dokumentit ja koko muutoshistoria säilyvät. TuntiTutkan repositoryssa ovat kansiot client/, server/ ja project-docs/.
commit vko 1	<i>yksi tallennettu muutos.</i> Git-historiaan tallennettu muutos ja sen viesti. Yksi commit on yksi looginen muutos, ja commit-historia on itsessään näyttöaineistoa.
frontend	<i>selaimessa toimiva osa.</i> Se osa sovelluksesta, jonka käyttäjä näkee ja jota hän klikkaa. TuntiTutkan frontend tehdään Sveltellä ja Vitellä kansioon client/.
backend	<i>palvelimella toimiva osa.</i> Se osa sovelluksesta, joka ajetaan palvelimella: rajapinta, käyttöoikeudet ja tietokantayhteys. TuntiTutkan backend tehdään Node.js:llä ja Expressillä kansioon server/.
p1, s1, k1	<i>näyttömatriisin vaatimustunnukset.</i> Viikkojen Näytä-riveillä olevat tunnukset viittaavat näyttömatriisin osaamisvaatimuksiin: p on Ohjelmointi, s on Ohjelmistokehittäjänä toimiminen ja k on ohjelmistokomponenttikirjasto. Numero yksilöi vaatimuksen, esimerkiksi s7.
P0 vko 2	<i>pakollinen ydin.</i> Ominaisuudet, joiden on valmistuttava ennen mitään lisäominaisuuksia: kirjautuminen ja roolit, tuntikirjaus, omat kirjaukset ja lasketut yhteenvedot. P0 tehdään ensin.
P1 vko 2	<i>tärkeä jatkosisältö.</i> Sisältö, joka tehdään vasta kun P0 toimii. TuntiTutkassa P1 on kaaviot, mobiilihionta ja yhteenvedojen vienti taulukkotiedostoksi.
P2 vko 2	<i>valinnainen lisä.</i> Sisältö, joka voidaan jättää kokonaan pois ilman että työ jää kesken. TuntiTutkassa P2 on kirjausmuistutus ja usean viikon vertailu.
GitHub-issue vko 2	<i>yhden muutoksen tehtäväkortti GitHubissa.</i> Yksi rajattu muutos sovellukseen: tavoite, hyväksymiskriteerit ja työmääräarvio. Yksi issue on tyypillisesti puolen tai yhden päivän työ, ja siihen viitataan numerolla, esimerkiksi #12. Sivun työvaihe voi sisältää useita issueita, ja jokainen issue tehdään Työtapa-sivun kuudella askeleella.
API vko 3	<i>sovelluksen rajapinta.</i> Palvelimen osoitteet, joita selain kutsuu tietojen hakemiseen ja tallentamiseen, esimerkiksi /api/kirjaukset. TuntiTutkan rajapinta tarkistaa oikeudet itse: käyttöliittymästä piilotettu nappi ei ole suojaus.
JSON vko 3	<i>tekstimuotoinen tietomuoto.</i> Tapa esittää tieto nimi–arvo-pareina tekstinä. TuntiTutkassa JSON on sekä yksi vertailtavista tietovarastoista (JSON-tiedosto) että muoto, jossa rajapinta palauttaa kirjaukset ja raportit selaimelle.
build vko 3	<i>valmiiksi käännetty tuotantoversio.</i> Komennolla tuotettu versio sovelluksesta, joka viedään palvelimelle. Kehityspalvelin ei ole build: julkaistava versio syntyy aina erikseen.
CSV	<i>taulukkotiedostomuoto.</i> Tekstitiedosto, jossa sarakkeet on erotettu toisistaan ja jonka taulukkolaskenta avaa sellaisenaan. TuntiTutkassa yhteenvedojen vienti CSV-tiedostoksi on P1-sisältöä.
middleware vko 4	<i>välikerros.</i> Palvelimen funktio, joka tarkistaa jokaisen pyynnön ennen kuin reitin oma käsittelijä ajetaan. TuntiTutkassa middleware tarkistaa kirjautumisen (401) ja roolin (403).
REST vko 5	<i>rajapinnan osoitetyyli.</i> Tapa rakentaa rajapinta niin, että osoite kertoo kohteen ja HTTP-metodi toiminnon: GET /api/kirjaukset hakee kirjaukset ja POST /api/kirjaukset lisää uuden.
CRUD vko 6	<i>luonti, luku, muokkaus ja poisto.</i> Tietueen neljä perustoimintoa. Projektien CRUD tarkoittaa, että projektipäällikkö voi luoda, listata, muokata ja poistaa projektit itse ilman että tietokantaan kosketaan käsin.
SQL vko 8	<i>tietokannan kyselykieli.</i> Kieli, jolla tietoa haetaan ja muokataan SQLite-kannassa. TuntiTutkan yhteenvedot lasketaan SQL-kyselyllä jokaisella pyynnöllä, eikä summia tallenneta mihinkään.
haara vko 11	<i>branch.</i> Oma kehityslinja, jossa muutoksen voi tehdä rikkomatta pääversiota. Työviikolla 11 palautemuutos tehdään omassa haarassa ja yhdistetään pääversioon vasta katselmoinnin jälkeen.

pull request vko 11	<i>PR, pyyntö yhdistää haara pääversioon.</i> GitHubissa avattava pyyntö, jossa haaran muutokset esitellään, katselmoidaan ja yhdistetään pääversioon. Kuvaus kertoo mitä muutettiin ja miksi. Lyhenne on PR.
XSS vko 13	<i>skriptin ujuttaminen syötteeseen.</i> Hyökkäys, jossa syötekenttään kirjoitettu skripti ajetaan toisen käyttäjän selaimessa. TuntiTutkassa testataan, että kirjauksen selitekenttään kirjoitettu skripti näkyy pelkkänä tekstinä.
T01 vko 14	<i>testitapauksen tunnus.</i> T tarkoittaa testitapausta ja numero yksilöi sen: T01 on ensimmäinen testitapaus ja T02 toinen. TuntiTutkassa tapauksia on 13 (T01–T13), ja jokaisen odotettu tulos kirjataan ennen ajoa.
RC vko 16	<i>release candidate, julkaisuehdokas.</i> Lähes valmis versio, joka julkaistaan ja testataan ennen lopullista julkaisua. TuntiTutkan ensimmäinen julkaisuehdokas on v1.0-rc1 työviikolla 16.
tagi vko 16	<i>versiomerkintä Git-historiassa.</i> Nimilappu, joka kiinnitetään tiettyyn committiin, jotta juuri se versio löytyy myöhemmin: v1.0-rc1 ja v1.0. Release on tagin ympärille tehty julkaisumerkintä kuvauksineen.

Näyttömatriisi – 32 osaamisvaatimusta

Rasti vasta, kun vaatimukselle on täsmällinen työnäyte: linkki, commit, kuva, testirivi tai muistio. Sama työnäyte voi kelpata useaan kohtaan.

Ohjelmointi

- ☐ **käyttää ohjelmointieditoria tai kehitysympäristöä** – VS Code, Vite dev -palvelin ja selaimen kehittäjätyökalut käytössä työviikolta 1: versiotaulukko, kuvakaappaus ja npm-skriptit README:ssä.
- ☐ **etsii ja korjaa virheitä ohjelmakoodista** – Kolme täydellistä virheenkorjausketjua (työviikot 14, 17; havainnot myös 8–9 ja 16): havainto, toisto, syy, korjauscommit, uusintatesti ja regressiotesti.
- ☐ **testaa ohjelman toimintoja** – Testiraportti työviikolta 14: vähintään 12 tapausta odotettuine tuloksineen ennen ajoa; lomakkeet, roolit, tallennus, virhetilanteet ja responsiivisuus testattuina.
- ☐ **käyttää rakenteista ohjelmointia toteutuksissa** – Laskentalogiikka omana moduulina työviikolla 8: funktiot, ehdot ja tietorakenteet raporttien ryhmittelyssä; moduulijako client, server, routes ja laskenta.
- ☐ **kirjoittaa ylläpidettävää ohjelmakoodia** – Kolme dokumentoitua refaktorointia työviikolla 15 ennen/jälkeen-diffeineen ja perusteluineen; nimeämiskäytäntö ja vastuiden jako kuvattuina.
- ☐ **tulkitsee suunnitelmia ja toteuttaa käyttöliittymän tai sen osia** – Kirjausnäkyvä toteutettu työviikolla 5 työviikon 2 rautalangan mukaan itse ilman valmista käyttöliittymäkirjastoa; mobiili ja palautteet käyttäjälle viimeistellään työviikolla 12.
- ☐ **tulkitsee suunnitelmia ja toteuttaa ohjelmiston toimintoja** – Hallintänäkymien toiminnot työviikolla 6 käyttäjätarinoiden ja hyväksymiskriteerien perusteella; omien kirjausten hallinta täydentää työviikolla 7. Issue-commit-ketju näkyvissä.
- ☐ **sopii tehtävistä tiimin muiden jäsenten kanssa** – Issue-taulu työviikolta 2 alkaen ja viikoittainen sopiminen ohjaajan kanssa tiimiroolissa; tehtävien tila näkyvänä koko projektin ajan.
- ☐ **etsii ratkaisuvaihtoehtoja ja ratkoo ongelmia yhdessä tiimin kanssa** – Istuntotapavertailu työviikolla 4: evästesession ja tokenin hyödyt ja riskit tässä sovelluksessa, kirjattu keskustelu ja yhteinen päätös ohjaajan kanssa.
- ☐ **arvioi ratkaisujen toimivuuden yhdessä tiimin kanssa** – Katselmointi työviikolla 10: täyttääkö toteutus käyttäjätarinat ja mitä muutetaan ennen seuraavaa versiota. Muistio ja priorisoitu muutoslista, todennus työviikolla 11.
- ☐ **arvioi omaa toimintaa tiimin jäsenenä** – Itsearviointi työviikolla 18 konkreettisin tilantein: mikä onnistui, missä tarvitsit apua ja keneltä, mitä tekisit toisin.

Ohjelmistokehittäjänä toimiminen

- ☐ **selvittää kehitystiimin kanssa asiakkaan tarpeet** – Toimeksianto purettu käyttäjätarinoiksi ja käyttäjäryhmiksi työviikolla 2; kysymyslista ja kirjatut vastaukset, rajausta sovittu ja täydennetty työviikolla 10.
- ☐ **viestii tekniset asiat asiakaslähtöisesti** – Katselmointiesittely työviikolla 10 ja luovutusviesti työviikolla 17 ilman teknistä jargonia: mitä ratkaisu tarkoittaa käyttäjälle, vaihtoehdot ja rajoitteet.
- ☐ **osallistuu version katselmointiin** – Kaksi katselmointia: väliversion asiakaskatselmointi työviikolla 10 ja julkaisuehdokkaan julkaisutestaus työviikolla 16. Palaute ja sovitut muutokset kirjattuina.
- ☐ **asettaa kehitystiimin kanssa toteutettavat toiminnot tärkeysjärjestykseen** – P0-, P1- ja P2-priorisointi ohjaajan kanssa työviikolla 2; P0-ydin eli kirjaus, roolit ja raportit toteutettu ensin.
- ☐ **jakaa kehitystiimin kanssa toteutettavat toiminnot tehtäviksi** – Käyttäjätarinat pilkottu issueiksi työviikolla 2 hyväksymiskriteereineen, noin puolen tai yhden päivän kokoisina; issue-taulu koko projektin ajan.
- ☐ **suunnittelee ja arvioi kehitystiimin kanssa tehtävien toteuttamista** – Työmääräarviot issueissa työviikolta 2 ja arvio vs. toteuma -vertailu työviikolta 7 omista TuntiTutka-kirjauksista; suunnitelman päivitys, kun arvio petti.
- ☐ **kehittää ohjelmiston toimintalogiikkaa** – Yhteenvetojen laskenta lennossa työviikolla 8 viikoittain, henkilöittäin ja tehtävälajeittain ilman tallennettuja summia; käyttöoikeudet, validointi ja omistajuussäännöt työviikoilta 4, 7 ja 13.
- ☐ **valitsee ohjelmistoon sopivan tietovaraston** – Vertailu SQLite, JSON-tiedosto ja PostgreSQL työviikolla 2 datan rakenteen, käyttötilanteen ja laajuuden perusteella; perusteltu valinta suunnitelmassa.
- ☐ **toteuttaa yhteyden tietovarastoon** – Kirjausten ja hallintadatan luku, lisäys, muokkaus ja poisto SQLitestä hallitusti työviikoilta 5–7; skeema ja init-skripti työviikolta 3.
- ☐ **hyödyntää rajapintoja ja käsittelee tietoa** – Raportti-API:n kutsu fetchillä työviikolla 9, JSON-muunnos kaavion muotoon testattuna funktiona ja virhetilanteet eli tyhjä data, verkkovirhe ja lataus käsiteltyinä.
- ☐ **arvioi ohjelmiston tietoturva** – Tietoturva-arvio työviikolla 13: syötteen, käyttöoikeudet, salasanat, istunnot ja tietojen näkyvyys; ajettu reitittaulukko ja XSS-testi, salaisuudet ympäristömuuttujissa.

- **käyttää versionhallintaa** – Git koko projektin ajan työviikolta 1: tarkoituksenmukaiset commitit, etärepository ja haarakäytäntö. Historia työnäytteenä, koonti työviikolla 11.
- **liittää ohjelman osan olemassa olevaan versioon** – Palautemuutos ominaisuushaarassa työviikolla 11: pull request, itsekselmointi, konfliktin ratkaisu ja merge pääversioon.
- **julkaisee ohjelman tuotantoympäristöön** – Tuotantobuild, ympäristöasetukset ja julkaisu valittuun pilvialustaan: ensijulkaisu työviikolla 3, julkaisuehdokas 16 ja v1.0 työviikolla 17 julkisessa osoitteessa.

Ohjelmiston toteuttaminen ohjelmistokomponenttikirjastolla

- **ottaa käyttöön ja konfiguroi ohjelmistokomponenttikirjaston käyttöön soveltuvan kehittämisympäristön** – Svelte- ja Vite-projektin luonti ja konfigurointi työviikolla 1: vite.config, dev-proxy backendiin työviikolla 3 sekä kehitys- ja tuotantoasetukset.
- **selvittää ohjelmistokomponenttikirjaston tarjoamat mahdollisuudet ja rajoitteet** – Kirjallinen selvitys työviikolla 3: mitä Svelte ja Vite ratkaisevat eli komponentit, reaktiivisuus ja build, mitä eivät eli reititys ja kaaviot, ja mistä puuttuva otetaan.
- **käyttää ohjelmistokomponenttikirjaston tärkeimpiä toimintoja ja työkaluja** – Komponentit, propsit, tapahtumat, lomakesidonat, ehdollinen renderöinti ja store istuntotilalle osoitettuina kirjausnäkyssä työviikolla 5 ja raporttinäkyssä 9.
- **tuo kehittämisympäristöön ulkoisia komponentteja** – Kaksi perusteltua ulkoista komponenttia: reitityskirjasto työviikolla 6 vertailuineen ja konfigurointeineen sekä kaaviokirjasto työviikolla 9 kokoineen, lisensseineen ja riippuvuuksineen.
- **suunnittelee, toteuttaa ja testaa ohjelmiston ohjelmistokomponenttikirjastoa käyttäen** – Komponenttirakenne ja vastuut suunnitelmassa työviikolla 2, sovellus toteutettu Sveltellä käyttäjätarinoiden mukaan ja omat sekä kirjastoon liittyvät ratkaisut testattuina työviikolla 14.
- **julkaisee ohjelmiston asiakkaan ympäristöön** – Viten tuotantobuild ja julkaisu sovittuun pilviympäristöön: julkaisuehdokas tagilla v1.0-rc1 työviikolla 16 ja v1.0 työviikolla 17.
- **dokumentoi ohjelmiston sovitulla tavalla** – README työviikolla 15: käyttöönotto, käynnistys, riippuvuudet rooleineen ja ympäristöasetukset; lisäksi asiakkaan pikaohjeet molemmille rooleille.

Muista: sivuston rastit ja kentät tallentuvat vain selaimeen. Ne eivät siirry opettajalle eivätkä korvaa Gitissä olevaa työtä.